

СОДЕРЖАНИЕ

Введение.....	7
1 Обзор литературы	8
1.1 Многоагентные системы на основе искусственного интеллекта	8
1.2 LangChain	9
1.3 LangGraph	10
1.4 Архитектурный паттерн Retrieval-Augmented Generation	12
1.5 Модели для семантического анализа и формирования векторных представлений	13
1.6 Векторные базы данных для хранения и поиска	15
1.7 Обзор аналогов	17
1.8 Технологии парсинга и обработки документов	20
2 Системное проектирование	22
2.1 Блок взаимодействия с пользователем	22
2.2 Блок обеспечения безопасности и контроля доступа	23
2.3 Блок агрегации и первичной обработки данных	23
2.4 Блок парсинга и извлечения сущностей	23
2.5 Блок генерации векторных представлений	24
2.6 Блок векторного хранилища и индексации	24
2.7 Блок гибридного поиска и ранжирования	24
2.8 Блок оркестрации и координации агентов	25
2.9 Общие принципы взаимодействия блоков	25
3 Функциональное проектирование	26
3.1 Анализ данных	26
3.2 Базовые классы моделей данных	27
3.3 Описание сервиса extraction_service	30
3.4 Описание сервиса storage_service	36
3.5 Описание сервиса search_service	39
3.6 Классы модуля orchestrator-service	43
3.7 Описание сервиса frontend	50
3.8 Описание сервиса auth_service	53
4 Разработка Программных модулей	56
4.1 Модули сервиса извлечения данных	56
4.2 Модули сервиса векторного хранилища и индексации	59
4.3 Модули сервиса семантического поиска	60
4.4 Модули сервиса оркестрации	62
4.5 Модули сервисов взаимодействия с пользователем и безопасности	65
4.6 Разделяемые библиотеки	67
5 Программа и методика испытаний	69
5.1 Подраздел	69
6 Руководство пользователя	70
6.1 Подраздел	70
7 Техничко-экономическое обоснование разработки системы подбора персонала с семантическим анализом агрегированных данных	71

7.1 Характеристика программного средства, разрабатываемого для реализации на рынке	71
7.2 Расчет инвестиций в разработку программного средства для его реализации на рынке	71
7.3 Расчет экономического эффекта от реализации программного средства на рынке	74
7.4 Расчет показателей экономической эффективности разработки и реализации программного средства на рынке	76
7.5 Вывод об экономической целесообразности реализации проектного решения.....	77
Заключение	78
Список использованных источников	79
Приложение А	81
Приложение Б	82
Приложение В.....	83
Приложение Г	84
Приложение Д.....	85

ВВЕДЕНИЕ

В условиях цифровой трансформации рынка труда и экспоненциального роста объемов данных о соискателях особую актуальность приобретает разработка интеллектуальных систем автоматизированного подбора персонала [1]. Современные рекрутинговые платформы, профессиональные сети и корпоративные базы данных формируют распределённые массивы информации о кандидатах, включающие резюме в различных форматах, профили в социальных сетях, результаты тестирований и рекомендательные письма.

Резюме и иные источники данных о кандидатах характеризуются высокой степенью неструктурированности, разнообразием лексических формулировок для описания одних и тех же компетенций (например, «machine learning engineer», «ML-разработчик», «специалист по машинному обучению»), а также наличием шумовых компонентов в виде маркетинговых формулировок и субъективных оценок. Традиционные системы поиска, основанные на ключевых словах и жёстких фильтрах, не способны учитывать семантическую близость понятий и контекстуальные связи между требованиями вакансии и опытом кандидата, что приводит к низкой релевантности результатов и увеличению трудозатрат рекрутеров [2]. Для повышения качества подбора персонала необходимо обеспечить агрегацию разнородных данных из множества источников с последующей их нормализацией и семантической интерпретацией.

Целью дипломного проектирования является разработка и реализация интеллектуальной системы подбора персонала с семантическим анализом агрегированных данных на основе многоагентной архитектуры, обеспечивающей автоматизированную обработку резюме, генерацию мультиязычных эмбеддингов и гибридный поиск релевантных кандидатов.

В соответствии с целью определены следующие задачи:

- анализ существующих подходов к семантическому поиску в рекрутинге и выбор архитектурных решений;
- проектирование многоагентной системы агрегации и нормализации данных о кандидатах;
- реализация модуля извлечения структурированной информации из резюме в различных форматах;
- интеграция эмбеддинг модели для генерации семантических векторных представлений профилей кандидатов;
- разработка гибридного механизма поиска, комбинирующего семантическое сходство и полнотекстовые фильтры;
- создание пользовательского интерфейса для взаимодействия рекрутера с системой и валидация результатов поиска.

Данный дипломный проект выполнен мной лично, проверен на заимствования, процент оригинальности составляет XX% (отчет о проверке на заимствования прилагается).

1 ОБЗОР ЛИТЕРАТУРЫ

1.1 Многоагентные системы на основе искусственного интеллекта

1.1.1 Многоагентные системы представляют собой класс архитектур искусственного интеллекта, в которых для решения сложных задач взаимодействуют несколько автономных программных сущностей – агентов [3]. Агент в данном контексте определяется как самостоятельная вычислительная единица, обладающая способностью воспринимать окружающую среду (в виде входных данных), анализировать полученную информацию, принимать решения и выполнять действия, направленные на достижение поставленной цели. Каждый агент может быть специализирован на выполнении ограниченного круга функций, а их совместная работа позволяет решать задачи, превышающие возможности отдельного агента или традиционной монолитной программы.

В отличие от последовательных алгоритмов или простых цепочек обработки, многоагентные системы характеризуются распределённостью, возможностью динамического планирования и наличием механизмов координации. Агенты обмениваются информацией, делегируют задачи и корректируют поведение в зависимости от промежуточных результатов. Такой подход особенно эффективен при обработке больших объёмов неструктурированных данных, требующих последовательного анализа, синтеза и принятия решений. В области подбора персонала многоагентные системы применяются для автоматизации этапов сбора данных о кандидатах, извлечения ключевых характеристик, семантического сопоставления профилей с требованиями вакансий и формирования ранжированного списка рекомендаций.

1.1.2 Основой функционирования современных агентов служат большие языковые модели (large language models, LLM) [4]. Большая языковая модель – это нейронная сеть, обученная на огромных массивах текстовых данных, способная глубоко понимать естественный язык, генерировать связные и грамматически правильные тексты, выполнять логические рассуждения, отвечать на вопросы, переводить, суммировать, писать код и решать широкий спектр задач, связанных с обработкой и генерацией текста.

В основе архитектуры практически всех современных больших языковых моделей лежит трансформер – структура, предложенная в 2017 году в статье «Attention Is All You Need» [5]. Трансформер отказался от рекуррентных слоёв (RNN, LSTM), которые обрабатывали последовательности шаг за шагом, и ввёл механизм самовнимания (self-attention). Этот механизм позволяет модели одновременно учитывать взаимосвязи между всеми токенами в последовательности, независимо от расстояния между ними. Самовнимание вычисляет, насколько каждый токен важен для понимания других токенов в контексте, присваивая им веса внимания. Это даёт возможность эффективно захватывать долгосрочные

зависимости (например, связь между подлежащим и сказуемым в длинном предложении) и обрабатывать текст параллельно, что значительно ускоряет обучение и инференс по сравнению с предыдущими архитектурами.

1.1.3 К 2026 году большие языковые модели эволюционировали в сторону разреженных архитектур, среди которых ключевое место занимает Mixture-of-Experts (MoE, смесь экспертов) [6]. В отличие от традиционных плотных моделей, где все параметры активируются для обработки каждого токена, в MoE-моделях общее количество параметров может быть очень большим, но для каждого входного токена активируется лишь небольшая доля из них – обычно от нескольких миллиардов до десятков миллиардов, что резко снижает вычислительные затраты при сохранении высокого качества. Примером служит семейство Qwen3/Qwen3.5 от Alibaba, которое сочетает MoE с механизмами линейного внимания, поддерживает нативную мультимодальность и режимы глубокого рассуждения и быстрого ответа [7]. Такие модели демонстрируют конкурентные результаты на бенчмарках в задачах рассуждения, кодирования, математики и агентивного поведения, часто превосходя более крупные плотные модели при значительно меньшем потреблении ресурсов.

1.2 LangChain

LangChain представляет собой открытую программную платформу, предназначенную для разработки приложений на основе больших языковых моделей [8]. Созданная в 2022 году, платформа к 2026 году прошла несколько крупных обновлений и стала одним из наиболее широко используемых инструментов для построения сложных AI-систем. LangChain предлагает модульный подход к интеграции больших языковых моделей с внешними источниками данных, инструментами и механизмами управления состоянием, что позволяет создавать приложения, выходящие за рамки простого генеративного текста. Основная цель платформы – упростить переход от прототипирования к производственным решениям за счёт унифицированных интерфейсов, надёжных абстракций и инструментов наблюдения за поведением системы.

1.2.1 Принцип модульности LangChain позволяет конструировать приложения из независимых переиспользуемых блоков. Ключевые компоненты включают модели, обеспечивающие унифицированный доступ к различным большим языковым моделям, включая локальные и облачные реализации; шаблоны запросов, стандартизирующие формирование входных данных для моделей с поддержкой переменных, примеров и динамического контекста; цепочки обработки, представляющие собой последовательные комбинации вызовов моделей, преобразований данных и инструментов; инструменты, реализующие внешние функции, такие как доступ к базам данных, парсинг файлов, вызов API или математические вычисления;

механизмы памяти для хранения краткосрочного и долгосрочного контекста; ретриверы, отвечающие за извлечение релевантной информации из векторных хранилищ с поддержкой семантического, ключевого и гибридного поиска. Агент в рамках LangChain реализует цикл рассуждения и действий: анализирует задачу, планирует последовательность вызовов инструментов, выполняет их и корректирует поведение на основе полученных результатов. Компоненты экосистемы LangChain отображены на рисунке 1.1.



Рисунок 1.1 – Компоненты экосистемы LangChain

1.2.2 В системах подбора персонала агенты, реализованные на платформе LangChain, применяются для автоматизированного извлечения структурированной информации из резюме и сопроводительных документов с использованием цепочек парсинга, анализа профессиональных компетенций через семантическое сравнение текстовых описаний навыков и опыта; предварительной фильтрации кандидатов на основе гибридного поиска по векторным и атрибутивным критериям; формирования ранжированного списка рекомендаций с объяснением релевантности. Для сложных многоагентных сценариев необходимо комбинировать LangChain с LangGraph, что обеспечит явное управление состоянием, ветвлениями и устойчивостью к ошибкам.

1.3 LangGraph

1.3.1 LangGraph является расширением экосистемы LangChain, предназначенное для построения управляемых состоянием многоагентных и многошаговых процессов в форме направленных графов [9]. В отличие от линейных цепочек (chains) и относительно простых агентов в базовом LangChain, LangGraph позволяет явно моделировать сложные рабочие процессы как граф, где каждый узел соответствует действию, вызову модели, инструменту или человеческому вмешательству, а направленные рёбра задают возможные переходы, включая условные ветвления, циклы, возвраты к

предыдущим состояниям и параллельное выполнение. Визуальное сравнение систем, построенных на LangChain и LangGraph отображено на рисунке 1.2.

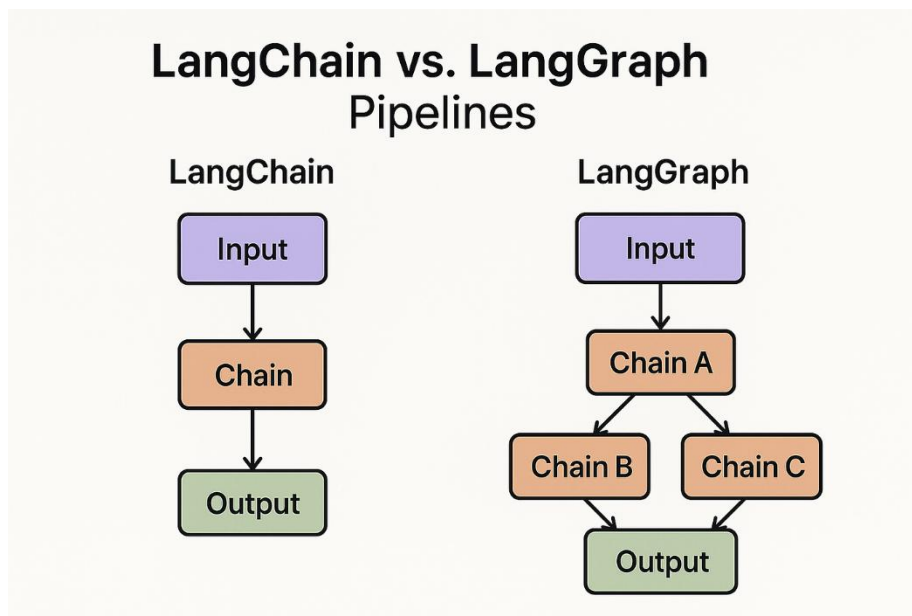


Рисунок 1.2 – Сравнение LangChain и LangGraph систем

Основная идея заключается в том, чтобы сделать поведение агентов детерминированным и отлаживаемым даже в условиях неопределённости и длительных последовательностей действий. Состояние графа (state) хранится централизованно и может быть персистентным – сохранённым между запусками, что критично для задач, требующих многочасовой или многодневной обработки.

1.3.2 Граф в LangGraph строится из узлов и рёбер. Узел может быть функцией, цепочкой, агентом или специальным блоком. Рёбра определяют не только последовательность, но и условия перехода: условные рёбра позволяют динамически выбирать следующий узел на основе текущего состояния или результата предыдущего шага. Состояние графа передаётся между узлами в виде объекта, который может содержать любые данные: от простых строк до сложных структур с историей сообщений, промежуточными результатами и метаданными.

Важные механизмами являются:

- сохранение промежуточных состояний (checkpointing);
- приостановка выполнения для получения ввода человека (human-in-the-loop);
- потоковая передача промежуточных результатов (streaming);
- асинхронное выполнение и встроенную обработку ошибок через retry-политики и fallback-узлы.

LangGraph поддерживает интеграцию с LangSmith для полной трассировки каждого пути выполнения графа, визуализации зависимостей и

сравнения различных вариантов workflow. Это позволяет проводить А/В-тестирование стратегий агентов и находить узкие места в реальном времени.

1.3.3 В системах интеллектуального подбора персонала LangGraph используется для формализации полного цикла обработки кандидата как управляемого графа. Типичный workflow может включать следующие узлы: сбор данных (парсинг резюме и профилей из внешних источников), извлечение и нормализация сущностей (опыт, навыки, образование), генерация векторных представлений, гибридный поиск по базе кандидатов, ранжирование с учётом дополнительных фильтров, генерация объяснения релевантности и, при необходимости, запрос уточнения у рекрутера.

1.4 Архитектурный паттерн Retrieval-Augmented Generation

1.4.1 Архитектурный паттерн Retrieval-Augmented Generation (RAG) представляет собой гибридный подход к построению систем искусственного интеллекта, который комбинирует возможности больших языковых моделей с внешними базами знаний [10]. Основная проблема традиционных языковых моделей заключается в ограниченности их знаний временным срезом данных на момент обучения и склонности к генерации фактологических ошибок, так называемых галлюцинаций. Паттерн RAG решает эту задачу путем динамического извлечения релевантной информации из внешнего источника перед генерацией ответа. В контексте интеллектуального подбора персонала это позволяет системе оперировать актуальными данными о кандидатах, хранящимися в векторной базе, а не полагаться исключительно на внутренние знания модели. Схема архитектурного паттерна приведена на рисунке 1.3.

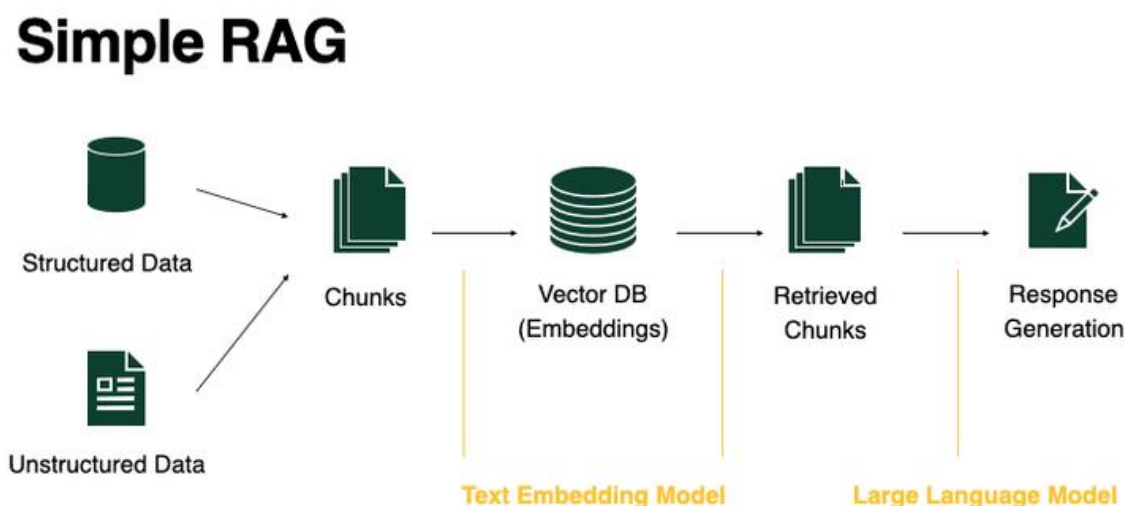


Рисунок 1.3 – Схема архитектурного паттерна RAG

Процесс работы систем с применением RAG архитектуры делится на три ключевых этапа. На этапе индексации документы разбиваются на смысловые

фрагменты, преобразуются в векторные представления и сохраняются в хранилище. На этапе поиска при поступлении запроса система формирует эмбединг запроса и находит наиболее близкие по семантике фрагменты в базе данных. На этапе генерации найденные фрагменты добавляются в контекст запроса к языковой модели, которая формирует итоговый ответ на основе предоставленных фактов.

1.4.2 Современное развитие паттерна RAG предполагает переход от наивного поиска к продвинутым стратегиям, таким как гибридный поиск и повторное ранжирование [11]. Гибридный поиск комбинирует семантическое сходство векторов с традиционным полнотекстовым поиском по ключевым словам, что позволяет учитывать как смысловую близость, так и точное совпадение терминов, например названий конкретных технологий или должностей. Механизм повторного ранжирования применяет более тяжелые и точные модели к отобранному короткому списку кандидатов, чтобы уточнить порядок выдачи результатов. Особое значение для многоагентных систем имеет концепция Agentic RAG, где агент самостоятельно планирует стратегию поиска, выбирает необходимые инструменты и источники данных, а также критически оценивает полученную информацию перед формированием ответа. Такой подход обеспечивает гибкость системы и позволяет обрабатывать сложные многошаговые запросы рекрутеров, требующие анализа различных аспектов профиля кандидата.

1.4.3 Качество работы системы Retrieval-Augmented Generation напрямую зависит от качества исходных данных, что делает задачи парсинга и обработки документов критически важными. Резюме и сопроводительные письма часто поступают в форматах, сложных для автоматической обработки, таких как PDF с многоколоночной версткой, таблицами, вложенными элементами и смешанным контентом. Традиционные методы извлечения текста, основанные на простых парсерах, часто теряют структурную информацию, нарушают логический порядок блоков или некорректно интерпретируют специальные символы. Для решения этих задач в разрабатываемой системе применяются современные инструменты документного интеллекта, обеспечивающие сохранение семантической структуры исходных материалов.

1.5 Модели для семантического анализа и формирования векторных представлений

1.5.1 Семантический анализ текстов в современных системах искусственного интеллекта опирается на преобразование естественного языка в числовые векторы фиксированной размерности – эмбединги (embeddings). Эти векторы размещаются в многомерном пространстве таким образом, что фрагменты текста, близкие по смыслу, оказываются геометрически близкими друг к другу. Мерой близости чаще всего выступает косинусное сходство или

евклидово расстояние, которое позволяет количественно оценивать семантическую релевантность даже при отсутствии общих лексических единиц, учитывая синонимию, контекстные оттенки и структурные особенности высказывания.

Эмбединги решают фундаментальную проблему несоответствия между дискретной природой слов и непрерывной природой смысла, позволяя машинам оперировать текстом на уровне абстрактных понятий. Эмбединги снижают размерность исходных данных: из тысяч или миллионов токенов формируется компактный вектор (обычно от 384 до 4096 измерений), который сохраняет основную семантическую информацию и может быть эффективно индексирован и сравнен.

1.5.2 Исторически эмбединги прошли три основных этапа развития. Первое поколение – частотные модели (TF-IDF, Bag-of-Words) – опиралось исключительно на статистику встречаемости слов в документе и корпусе, полностью игнорируя семантику и порядок. Второе поколение – предсказательные модели (Word2Vec с архитектурами CBOW и Skip-gram, GloVe, FastText) – использовало нейронные сети для обучения статических векторов на основе локального контекста или глобальной кооккурренции слов [12]. Эти модели впервые позволили захватывать семантические отношения, но оставались статичными: каждое слово получало один и тот же вектор независимо от окружающего контекста.

Третье поколение – контекстуальные модели на базе трансформеров – радикально изменило ситуацию. BERT ввёл двунаправленное внимание и предобучение по маскированным токенам, позволяя генерировать разные представления одного и того же слова в зависимости от предложения [13]. Последующие модели (RoBERTa, ELECTRA, T5, DeBERTa) улучшали качество за счёт большего объёма данных, более сложных задач предобучения и оптимизаций. Современные модели, оптимизированные специально для задач поиска и retrieval (E5, BGE, GTE, Snowflake Arctic Embed), используют контрастивное обучение на парах (positive и negative примеры), чтобы максимизировать разделимость релевантных и нерелевантных документов в векторном пространстве.

1.5.3 Для задач, требующих обработки текстов на нескольких языках применяются мультязычные модели эмбедингов. Они обучаются на параллельных или смешанных многоязычных корпусах и формируют единое векторное пространство, в котором документы на разных языках могут сравниваться напрямую без предварительного перевода.

Семейство Qwen3-Embedding, разработанное на базе фундаментальных моделей Qwen3/Qwen3.5, занимает лидирующие позиции среди открытых мультязычных решений. Модели серии (0,6 млрд, 4 млрд и 8 млрд) демонстрируют высокие результаты на многоязычном рейтинге MTEB, особенно в задачах поиска по смыслу (retrieval), оценки семантической близости текстов (semantic textual similarity) и классификации

[14]. Qwen3-Embedding поддерживает более 100 языков, включая русский, обладает длиной контекста до 32 тысяч токенов, умеет учитывать инструкции (instruction-aware) и применяет технологию Matryoshka Representation Learning, которая обеспечивает гибкую размерность выходных векторов. Эти характеристики делают данное семейство моделей оптимальным выбором для проекта, в котором необходимо точно сопоставлять резюме и описания вакансий на разных языках.

1.6 Векторные базы данных для хранения и поиска

1.6.1 Векторные базы данных предназначены для хранения и быстрого поиска в массивах высокомерных векторов, представляющих семантические представления текстов, изображений или других объектов. В отличие от традиционных реляционных или документных баз, где поиск выполняется по точному совпадению или по ключевым словам, векторные базы ориентированы на приближённый поиск ближайших соседей (approximate nearest neighbors, ANN). Это позволяет находить наиболее семантически близкие записи даже при отсутствии буквального совпадения.

Основной алгоритмический приём – использование индексов приближённого поиска, среди которых наиболее распространён HNSW (Hierarchical Navigable Small World) [15], схема приведена на рисунке 1.4. Этот метод строит многоуровневый граф, где каждый узел (вектор) связан с ближайшими соседями на разных уровнях иерархии. Поиск начинается с верхнего уровня и постепенно уточняется на нижних, что обеспечивает баланс между скоростью и точностью.

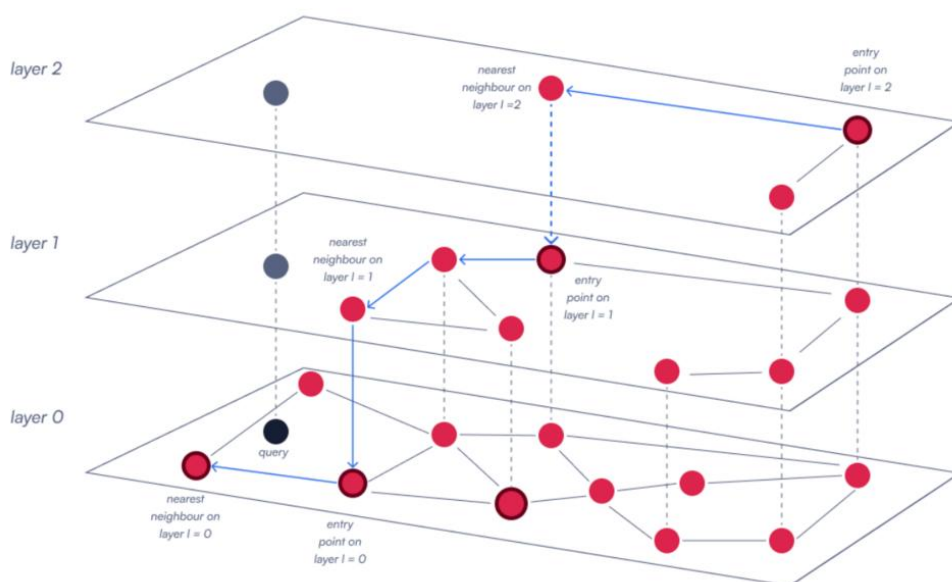


Рисунок 1.4 – HNSW алгоритм

Векторные базы также поддерживают хранение сопутствующих метаданных (payload), фильтрацию по атрибутам до или после поиска и

гибридный режим, когда векторный поиск сочетается с традиционным полнотекстовым (BM25, TF-IDF) или структурным. Это делает их незаменимыми в задачах, где семантика является лишь одним из критериев релевантности.

1.6.2 Qdrant – это высокопроизводительная открытая векторная база данных, написанная на языке Rust и ориентированная на низкую задержку и высокую пропускную способность [16]. Qdrant остаётся одним из лидеров среди открытых решений благодаря сочетанию скорости, надёжности и удобства интеграции с экосистемой Python и LangChain.

База использует собственную реализацию HNSW с оптимизациями для кэширования, сжатия векторов и параллельного выполнения запросов. Qdrant поддерживает хранение произвольных метаданных в формате `payload`, что позволяет прикреплять к каждому вектору профиль кандидата: опыт работы, навыки, желаемая зарплата, регион, уровень владения языками. Фильтрация по этим полям может выполняться как до поиска (*pre-filtering*), так и после (*post-filtering*), что критично для HR-задач, где нужно соблюдать жёсткие ограничения. Графический интерфейс с отображением векторов и полезной нагрузки отображен на рисунке 1.5.

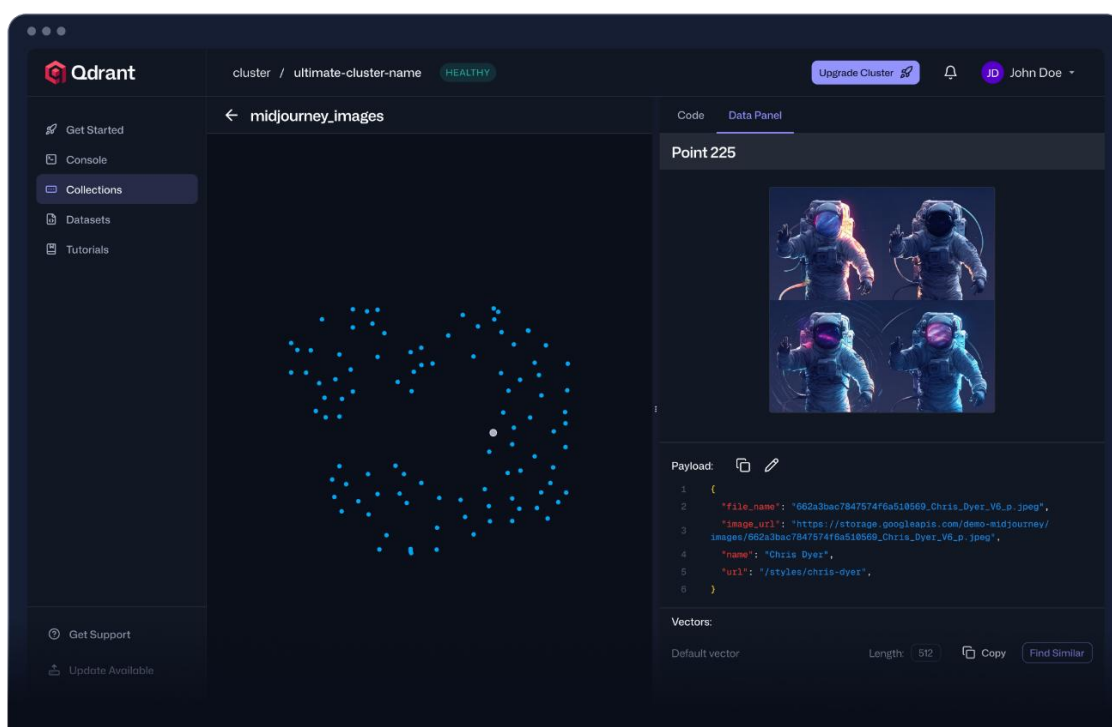


Рисунок 1.5 – Графический интерфейс Qdrant

Гибридный поиск в Qdrant объединяет векторную близость с полнотекстовым индексом, позволяя ранжировать результаты по комбинированной метрике. Дополнительные возможности включают горизонтальное масштабирование за счёт кластеризации, репликацию данных, создание моментальных снимков состояния базы в определённый момент

времени (point-in-time snapshots) и встроенную поддержку интерфейсов REST и gRPC. Интеграция с платформой LangChain выполняется через официальный адаптер хранилища векторов (vectorstore-адаптер), что существенно упрощает применение базы в многоагентных рабочих процессах (workflow).

1.6.3 В проекте интеллектуальной системы подбора персонала Qdrant выполняет роль центрального хранилища профилей кандидатов. После парсинга резюме и формирования векторных представлений с помощью модели Qwen3-Embedding каждый профиль индексируется как точка (point) с соответствующим payload. При поступлении запроса (описание вакансии) система генерирует вектор запроса и выполняет гибридный поиск: семантическая близость определяет основных кандидатов, а фильтры по атрибутам отсекают неподходящих.

Возможность обновления и удаления точек в реальном времени обеспечивает актуальность базы при поступлении новых резюме или изменении статуса кандидата.

1.7 Обзор аналогов

Системы подбора персонала с применением искусственного интеллекта представляют собой комплексные платформы, которые интегрируют семантический анализ, предиктивную аналитику и автоматизацию взаимодействия с кандидатами. Они ориентированы на крупные и средние организации, где требуется обработка больших объёмов заявок и поддержка полного цикла найма.

1.7.1 Resume Matcher – активно развивающийся открытый проект, предназначенных для сравнения резюме с описанием вакансии [17]. Инструмент выполняет двухуровневый анализ: сначала по ключевым словам и совпадению терминов, затем по семантической близости с использованием современных моделей векторных. Результаты визуализируются в виде тепловой карты совпадений, процентного рейтинга релевантности и списка недостающих или избыточных компетенций. Проект также предлагает рекомендации по доработке резюме для повышения вероятности прохождения автоматических фильтров ATS.

Решение ориентировано как на соискателей (помогает оптимизировать резюме под конкретную вакансию), так и на небольшие команды рекрутеров, которым требуется быстрый и бесплатный инструмент локального анализа. Преимущества включают простоту установки (часто через Docker или pip), полную прозрачность кода, отсутствие зависимости от облачных сервисов и возможность доработки под конкретные языки или домены. Ограничения – отсутствие встроенной базы кандидатов, многоагентной логики и масштабируемости для больших объёмов данных. Интерфейс продукта приведен на рисунке 1.6.

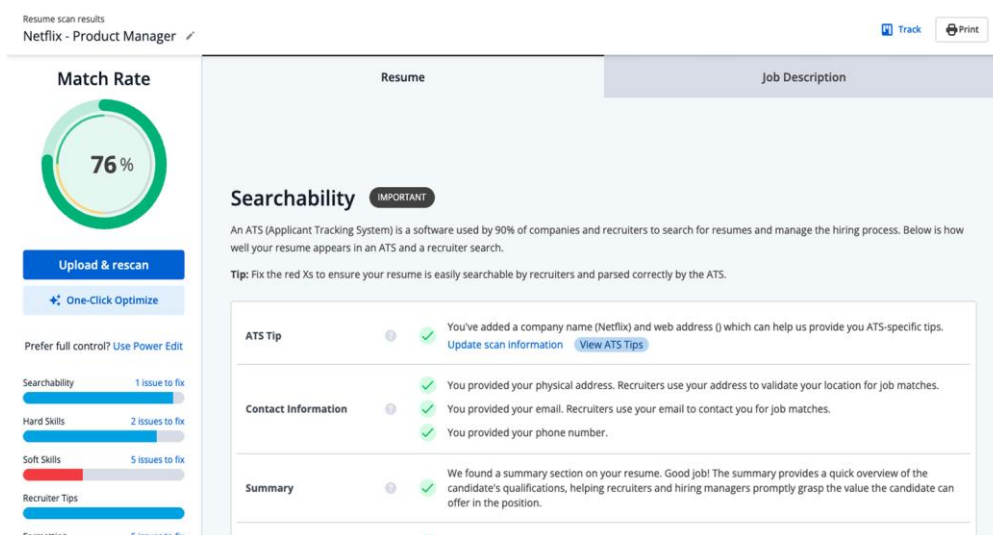


Рисунок 1.6 – Интерфейс Resume Matcher

1.7.2 Eightfold AI – одна из ведущих платформ консалтинговой компании Talent Intelligence, которая строит глубокие профили кандидатов на основе их навыков, профессионального опыта, образования и карьерных траекторий [18]. Система использует графовые представления знаний, где каждый кандидат связан с тысячами навыков, проектов и ролей. Это позволяет выполнять точное семантическое сопоставление с вакансиями, прогнозировать вероятность успеха на позиции, а также оценивать потенциал внутренней мобильности сотрудников. Интерфейс продукта представлен на рисунке 1.7.

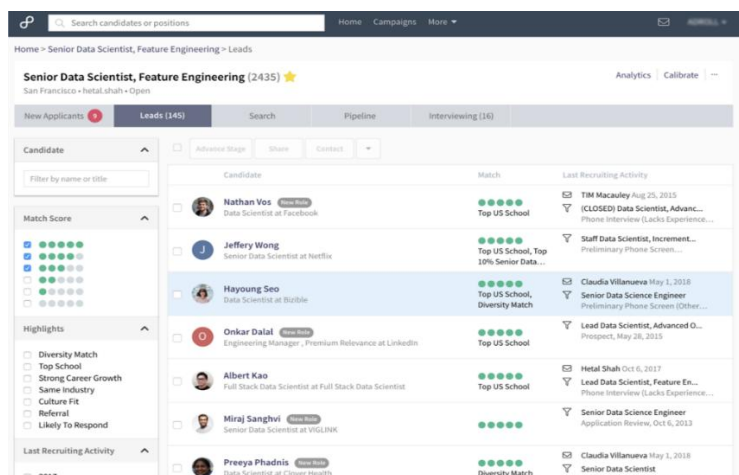


Рисунок 1.7 – Интерфейс Eightfold AI

Платформа включает механизмы автоматического снижения предвзятости, интеграцию с большинством популярных ATS и HRIS-систем, а также инструменты аналитики подбора. Eightfold особенно эффективна в крупных корпорациях, где требуется единый взгляд на весь пул талантов компании. Основные преимущества – высокая точность матчинга и поддержка

как внешнего набора, так и развития внутренних сотрудников. Ограничения связаны с высокой стоимостью внедрения и лицензирования, а также с частичной закрытостью алгоритмов. Интерфейс продукта приведен на рисунке 1.7.

1.7.3 Phenom – комплексная платформа Talent Experience Management, охватывающая весь жизненный цикл взаимодействия с кандидатами и сотрудниками [19]. Система включает создание персонализированных карьерных сайтов, чат-ботов для первичного общения, автоматизированный скрининг резюме, интеллектуальное планирование собеседований и аналитику вовлечённости на всех этапах. Семантический матчинг сочетается с предиктивными моделями, которые оценивают не только текущее соответствие, но и долгосрочный потенциал кандидата. Интерфейс продукта представлен на рисунке 1.8.

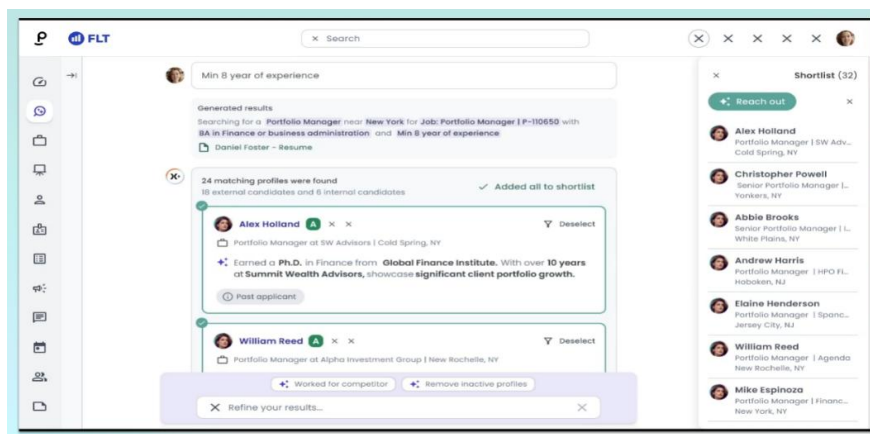


Рисунок 1.8 – Интерфейс Phenom

Phenom особенно сильна в сценариях высоконагруженного найма (розничная торговля, логистика, колл-центры, гостиничный бизнес), где ежедневно поступают тысячи заявок. Платформа активно использует персонализацию коммуникаций и автоматизацию рутинных задач рекрутеров. Преимущества – удобный пользовательский интерфейс, встроенная аналитика и поддержка полного цикла talent journey. Недостатки – высокая стоимость и необходимость значительной настройки под корпоративные процессы.

1.7.4 Коммерческие платформы обеспечивают готовое решение с высокой степенью автоматизации, поддержкой и соответствием корпоративным стандартам, однако требуют значительных финансовых вложений и ограничивают доступ к внутренней логике алгоритмов. Open-source инструменты, такие как Resume Matcher, предлагают гибкость и отсутствие затрат, но не покрывают полный цикл обработки и требуют самостоятельной доработки.

Разрабатываемая система занимает промежуточное положение: она использует открытые компоненты (LangChain, LangGraph, Qdrant, Qwen3-

Embedding), обеспечивая высокую кастомизацию, мультиязычность и прозрачность всех этапов. При этом проект реализует полноценный многоагентный рабочий процесс с семантическим анализом, гибридным поиском и возможностью человеческого контроля, что позволяет достичь уровня функциональности, сопоставимого с коммерческими решениями, при существенно меньших затратах и полной независимости от внешних поставщиков.

1.8 Технологии парсинга и обработки документов

1.8.1 Эффективность работы интеллектуальной системы подбора персонала напрямую зависит от качества извлечения информации из исходных документов. Резюме и сопроводительные письма поступают в систему в разнородных форматах. Каждый формат имеет свои особенности хранения структуры и метаданных. Файлы PDF могут быть как текстовыми, так и графическими, содержать многоколоночную верстку, таблицы и встроенные изображения. Документы DOCX хранят структуру в виде XML-тегов, что облегчает доступ к стилям и разделам, но требует специфических парсеров. Задача обработки заключается не только в извлечении текстового содержимого, но и в сохранении логической структуры документа, такой как заголовки разделов, списки навыков, хронология опыта работы и контактная информация. Потеря структурной информации на этапе парсинга приводит к ошибкам в последующей векторизации и семантическом поиске, поэтому выбор инструментария является критическим этапом проектирования.

1.8.2 Среди традиционных программных библиотек для работы с документами широко распространены инструменты с детерминированной логикой извлечения. Для формата PDF популярным решением является библиотека `pdfplumber` [20], которая строит геометрическую модель страницы на основе координат текстовых блоков. Этот подход позволяет точно определять границы колонок и таблиц, что критично для резюме со сложной версткой. Преимуществом таких решений является высокая скорость работы и низкие вычислительные затраты, так как не требуется запуск тяжелых нейронных сетей. Для формата DOCX стандартным инструментом выступает `python-docx`, обеспечивающий доступ к параграфам, стилям и таблицам через объектную модель документа. Однако детерминированные методы имеют существенное ограничение: они плохо справляются с неструктурированными данными, сканированными копиями документов или файлами с нарушенной кодировкой. В таких случаях извлеченный текст может быть фрагментирован или лишен смысловых связей между элементами.

1.8.3 В последнее время набирают популярность решения класса Document AI, использующие модели машинного обучения для понимания макета документа. Одним из современных представителей этого класса является библиотека `Docling` [21], разработанная для анализа сложных

документов с сохранением иерархии элементов, схема обработки представлена на рисунке 1.9. Инструмент применяет комбинацию компьютерного зрения и языкового моделирования для распознавания заголовков, абзацев, списков и таблиц, преобразуя их в структурированный формат, удобный для дальнейшей обработки. Аналогичный подход реализован в модели LayoutLM, которая обучается на совместном представлении текста и визуальных признаков страницы [22]. Такие технологии демонстрируют высокую устойчивость к вариативности шаблонов резюме и способны корректно интерпретировать данные даже при отсутствии явной разметки. Недостатком данных решений является повышенная требовательность к вычислительным ресурсам и большее время обработки одного документа по сравнению с традиционными парсерами.

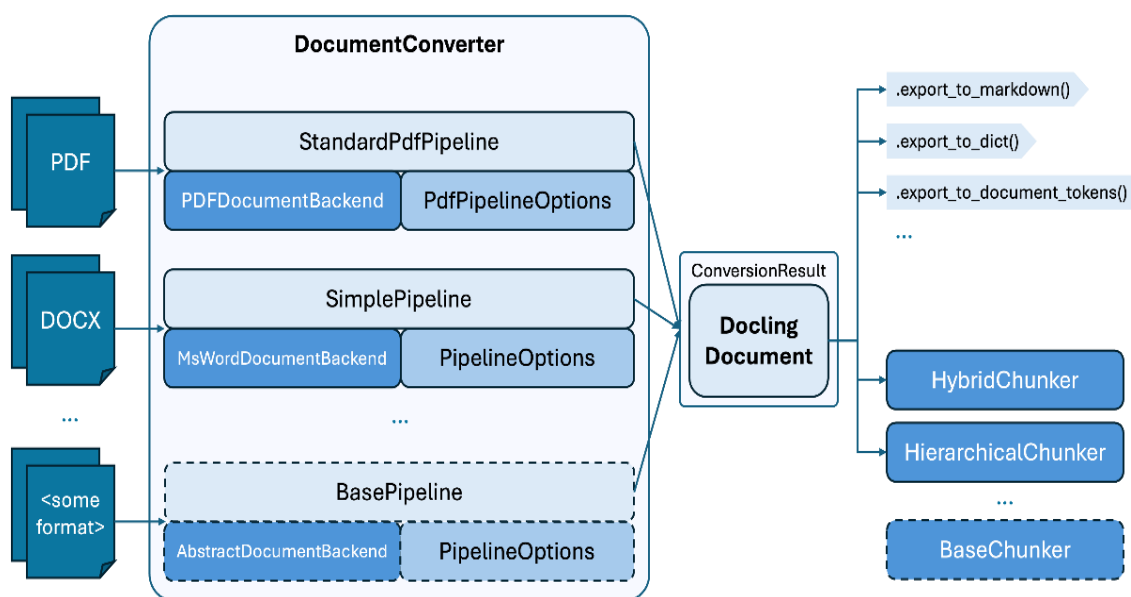


Рисунок 1.9 – Схема Docling

1.8.4 Существует также класс универсальных конвейеров обработки, таких как фреймворк Unstructured, который объединяет множество стратегий парсинга в единый интерфейс [23]. Подобные системы автоматически определяют тип файла и выбирают оптимальный метод извлечения, переключаясь между детерминированными и нейросетевыми подходами в зависимости от сложности документа. Это позволяет обеспечить баланс между производительностью и точностью. Для задач интеллектуального подбора персонала, где важна максимальная полнота данных о кандидате, наиболее перспективным представляется гибридный подход. Он предполагает использование легких библиотек для стандартных случаев и подключение тяжелых моделей анализа макета для сложных документов. Такой подход позволяет минимизировать потери информации на этапе первичной обработки и обеспечить высококачественную подготовку данных для генерации векторных представлений.

2 СИСТЕМНОЕ ПРОЕКТИРОВАНИЕ

В разработке интеллектуальной системы подбора персонала ключевым этапом является проектирование архитектуры, которая должна обеспечивать автоматическую обработку неструктурированных данных, глубокое семантическое понимание компетенций кандидатов и высокоточную выдачу наиболее релевантных соискателей. Процесс системного проектирования включает определение основных функциональных блоков, чёткое распределение ответственности между ними, установление взаимосвязей и способов взаимодействия компонентов.

После всестороннего анализа требований и задач дипломного проекта было принято решение разделить систему на следующие логические блоки:

- блок взаимодействия с пользователем;
- блок обеспечения безопасности и контроля доступа;
- блок агрегации и первичной обработки данных;
- блок парсинга и извлечения сущностей;
- блок генерации векторных представлений;
- блок векторного хранилища и индексации;
- блок гибридного поиска и ранжирования;
- блок оркестрации и координации агентов.

Структурная схема с перечисленными блоками представлена на чертеже ГУИР.400201.016 С1. Каждый блок подробно рассматривается в отдельном пункте.

2.1 Блок взаимодействия с пользователем

Блок взаимодействия с пользователем представляет собой интерфейсный слой системы, через который осуществляется всё общение кадрового специалиста с интеллектуальной системой подбора персонала. Его главная задача в обеспечении максимально удобного и интуитивно понятного процесса работы с системой на всех этапах: от формирования запроса до получения и анализа результатов.

Основные функции блока включают приём текстового запроса в свободной формулировке на естественном языке, применение дополнительных фильтров по опыту работы, уровню заработной платы, локации, обязательным и желательным компетенциям, загрузку новых резюме как в единичном, так и в пакетном режиме, а также отображение ранжированного списка кандидатов с визуализацией степени их соответствия вакансии и пояснениями причин релевантности. Кроме того, блок предоставляет возможность экспорта результатов в удобных форматах для дальнейшего использования.

Блок взаимодействия с пользователем передаёт сформированный запрос и загруженные документы в блок оркестрации и координации агентов, а затем получает от него полностью обработанные и структурированные результаты для их последующей визуализации и представления пользователю.

2.2 Блок обеспечения безопасности и контроля доступа

Блок обеспечения безопасности и контроля доступа реализует комплекс механизмов, направленных на защиту системы и всех обрабатываемых в ней данных. Он отвечает за аутентификацию, авторизацию и аудит действий, обеспечивая надёжную защиту от несанкционированного доступа на всех уровнях системы.

В основе работы блока лежит модель сервисных токенов, которые используются для подтверждения полномочий при взаимодействии между компонентами. Полученные токены передаются во все остальные блоки и позволяют осуществлять строгий контроль доступа к операциям в зависимости от роли и идентификатора вызывающего сервиса, а также ведут детальный аудит ключевых событий.

Благодаря данному блоку в системе формируется единый, целостный контур безопасности, который охватывает все компоненты и процессы. Это позволяет гарантировать конфиденциальность обрабатываемых персональных данных и предотвращать любые попытки несанкционированного использования функциональности системы.

2.3 Блок агрегации и первичной обработки данных

Блок агрегации и первичной обработки данных выступает в роли входного шлюза системы, через который поступают все документы для последующего анализа. Его задача заключается в обеспечении унификации и предварительной подготовки разнородных входных данных перед их глубоким структурным разбором.

Ключевые функции блока включают приём файлов резюме в различных форматах, определение типа документа и языка, нормализацию кодировок, очистку от очевидных артефактов, первичную категоризацию материалов и их маршрутизацию в следующий этап обработки. После завершения первичной обработки подготовленные данные направляются в блок парсинга и извлечения сущностей.

Правильная работа данного блока является необходимым условием для эффективной и стабильной работы всей системы, поскольку именно здесь закладывается основа качества дальнейшего семантического анализа.

2.4 Блок парсинга и извлечения сущностей

Блок парсинга и извлечения сущностей выполняет глубокий анализ неструктурированного текстового содержимого документов с целью создания формализованного профиля кандидата. Этот блок является одним из ключевых элементов системы, отвечающих за переход от необработанных данных к структурированной информации.

Основные задачи блока заключаются в распознавании структуры документа, извлечении именованных сущностей и атрибутов, нормализации

компетенций к единому контролируемому словарю, формировании строго типизированной модели профиля кандидата, а также обнаружении и обработке противоречий и неполноты данных. Полученный структурированный профиль передаётся в блок генерации векторных представлений и в блок оркестрации для дальнейшей обработки.

2.5 Блок генерации векторных представлений

Блок генерации векторных представлений отвечает за преобразование текстового и структурированного содержимого профиля кандидата в числовые векторные эмбединги, которые сохраняют семантическую близость информации. Данный блок обеспечивает возможность последующего сравнения кандидатов не только по формальным признакам, но и по смысловому соответствию.

Основные функции включают генерацию векторных представлений как всего профиля в целом, так и отдельных его значимых секций. Полученные эмбединги вместе с сопутствующими метаданными передаются в блок векторного хранилища и индексации, а при необходимости возвращаются в блок оркестрации и координации агентов.

2.6 Блок векторного хранилища и индексации

Блок векторного хранилища и индексации предназначен для долговременного хранения векторных представлений профилей кандидатов совместно с ассоциированными метаданными. Он обеспечивает эффективный и быстрый доступ к накопленным данным для выполнения поисковых операций.

Ключевые функции блока включают хранение векторов и связанных с ними данных, поддержку быстрого приближённого поиска по сходству, индексацию и фильтрацию по структурированным метаданным, а также инкрементальное обновление и версионирование профилей. Блок получает данные от блока генерации векторных представлений и предоставляет доступ к индексу для блока гибридного поиска и ранжирования.

2.7 Блок гибридного поиска и ранжирования

Блок гибридного поиска и ранжирования выполняет основную поисковую логику системы, объединяя несколько типов поиска и многоступенчатое ранжирование для достижения максимальной релевантности результатов. Именно этот блок отвечает за итоговое качество подбора кандидатов.

Основные механизмы включают первичный отбор кандидатов, применение дополнительных моделей переранжирования при необходимости, скоринг с учётом бизнес-правил и формирование финального упорядоченного списка с оценками степени соответствия. Блок получает запрос от блока

оркестрации, взаимодействует с блоком векторного хранилища и возвращает результаты через блок оркестрации в блок взаимодействия с пользователем.

2.8 Блок оркестрации и координации агентов

Блок оркестрации и координации агентов является центральным управляющим компонентом всей системы. Он реализует динамическое планирование и исполнение многошаговых процессов обработки запросов, обеспечивая адаптивность системы к различным сценариям работы.

Основные функции блока включают анализ входного запроса и определение оптимального сценария обработки, диспетчеризацию задач специализированным блокам, условную маршрутизацию, управление состоянием процесса, обработку исключительных ситуаций, агрегацию промежуточных результатов и формирование финального ответа. Блок получает начальный запрос от блока взаимодействия с пользователем, координирует работу всех остальных компонентов системы и возвращает полностью обработанный результат обратно в интерфейсный слой.

Таким образом, блок оркестрации обеспечивает целостность и логическую связность всей архитектуры, реализуя многоагентный подход к решению сложных задач подбора персонала.

2.9 Общие принципы взаимодействия блоков

Взаимодействие блоков системы построено на принципах модульности и централизованной координации. Каждый компонент отвечает за строго ограниченную задачу, а все ключевые процессы управляются через блок оркестрации и координации агентов. Он анализирует запрос, распределяет задачи по специализированным блокам, собирает промежуточные результаты и формирует финальный ответ, минимизируя прямые связи между остальными компонентами и упрощая трассировку и управление.

Обработка начинается с интерфейсного блока, откуда запрос поступает в оркестратор. Далее следует первичная очистка и унификация данных, парсинг и структурирование, генерация векторных представлений, сохранение в индексированном хранилище. При поиске оркестратор передаёт задачу блоку гибридного поиска и ранжирования, который обращается к хранилищу, комбинирует разные подходы и возвращает ранжированный список кандидатов с пояснениями.

Такая структура обеспечивает гибкость, предсказуемость и масштабируемость: логика процессов легко меняется в оркестраторе, блоки остаются независимыми. В результате система эффективно справляется с разнообразными задачами при высокой вариативности данных и требований.

3 ФУНКЦИОНАЛЬНОЕ ПРОЕКТИРОВАНИЕ

В данном разделе описываются структура разрабатываемого программного обеспечения с точки зрения данных и обрабатывающих их подпрограмм. В частности, рассматриваются модели данных, соответствующие им сущности и их взаимодействие, а также структура проекта.

Помимо функциональной структуры, важной частью разработки программного обеспечения является архитектурная структура. Архитектурная структура определяет общую организацию программы, ее компоненты и способы взаимодействия между ними. Она описывает, как различные части программного обеспечения взаимодействуют друг с другом и как они организованы для достижения целей приложения.

Система реализована в виде пяти микросервисов на языке Python с использованием фреймворка FastAPI и одного frontend-сервиса на TypeScript с использованием фреймворка NestJS и архитектурой микрофронтон Module Federation. Каждый сервис отвечает за строго определённую часть функционала и построен по принципам микросервисной архитектуры. Внедрение зависимостей и конфигурация осуществляются через singleton-паттерн Pydantic Settings. Логирование выполняется через structlog, корреляция запросов – через CorrelationMiddleware. Все сервисы общаются по HTTP с использованием JWT-токенов (через auth-service) и поддерживают асинхронную обработку.

Функциональная структура состоит из следующих микросервисов:

- `extraction_service` – блок агрегации, первичной обработки, парсинга и генерации векторных представлений;
- `storage_service` – блок векторного хранилища и индексации;
- `search_service` – блок гибридного поиска и ранжирования;
- `orchestrator_service` – блок оркестрации и координации агентов (центральный мозг системы);
- `frontend` – блок взаимодействия с пользователем;
- `auth_service` – блок обеспечения безопасности и контроля доступа.

Диаграмма всех классов приложения представлена на чертеже ГУИР.400201.016 PP.1.

3.1 Анализ данных

В качестве входных данных система принимает два основных типа запросов: загрузку резюме (файл) и поисковый запрос рекрутера. Входные данные обоих типов проходят валидацию на границе системы: файлы проверяются по MIME-типу и размеру, поисковые запросы – по схеме Pydantic-модели `SearchRequest`. Явное разграничение допустимых форматов и обязательных заголовков на уровне требований позволяет реализовать единообразную обработку ошибок во всех сервисах через стандартные классы

исключений `BadRequestError` и `ValidationError`, не дублируя логику проверки в каждом обработчике. Требования ко входным данным приложения представлены в таблице 3.1.

Таблица 3.1 – Требования ко входным данным

Категория	Требование
Формат файла	PDF, DOCX, DOC, PNG, JPG, JPEG (максимальный размер — 20 МБ)
Заголовки	X-Correlation-ID (обязательный), Authorization: Bearer <token>
Поисковый запрос	JSON-объект <code>SearchRequest</code> (<code>query</code> , <code>filters</code> , <code>top_k</code> , <code>use_rerank</code>)
Аутентификация	Валидный JWT-токен (роли: <code>recruiter</code> , <code>admin</code> , <code>system</code>)

В качестве выходных данных система формирует структурированные JSON-ответы. Требования к выходным данным представлены в таблице 3.2.

Таблица 3.2 – Требования к выходным данным

Категория	Требование
Формат ответа	<code>ExtractionResponse</code> / <code>SearchResponse</code> / <code>OrchestratorResponse</code> (Pydantic)
Структура профиля	<code>StructuredDocument</code> + <code>CandidateProfile</code> + чанки + эмбединги
Формат поиска	<code>RankedCandidate[]</code> с <code>final_score</code> , <code>matched_skills</code> , <code>explanation</code>
Кодировка	UTF-8
Real-time	WebSocket-события по <code>correlation_id</code> (обновления статуса)

Для поиска используется текстовый запрос на естественном языке с дополнительными фильтрами по навыкам, опыту, зарплате и локации. Разделение типов входных данных на два независимых пайплайна – загрузку и поиск – позволяет масштабировать каждый из них отдельно в зависимости от нагрузки.

3.2 Базовые классы моделей данных

Все сервисы используют общие Pydantic-модели v2 из реализованной библиотеки `libs/common`. Базовые классы обеспечивают единообразие структур данных, валидацию и сериализацию.

3.2.1 Класс `ExperienceEntry` (файл `libs/common/models/candidate.py`) является вложенной Pydantic-моделью, описывающей один период трудовой деятельности кандидата.

Поле `company` типа `str` хранит название организации и не может быть пустым – это обеспечивается валидатором `non_empty_str`, который обрезает пробелы и выбрасывает ошибку при пустой строке.

Поле `position` типа `str` хранит должность кандидата и проходит ту же валидацию, что и `company`. Поле `start_date` типа `Optional[str]` хранит дату начала работы в формате `YYYY` или `YYYY-MM`; допускает значение `None`.

Поле `end_date` типа `Optional[str]` хранит дату окончания в том же формате.

Поле `description` типа `Optional[str]` содержит произвольное текстовое описание обязанностей.

Поле `technologies` типа `list[str]` содержит список технологий, применявшихся на данной позиции; валидатор `strip_technologies` фильтрует пустые строки и обрезает пробелы.

Валидатор `validate_date_str` проверяет формат дат с помощью регулярного выражения `^\d{4}(-\d{2})?$.`

Валидатор модели `end_after_start` проверяет, что `end_date` не предшествует `start_date`, и выбрасывает `ValueError` при нарушении хронологии.

3.2.2 Класс `SkillEntry` описывает одиночный навык кандидата.

Поле `name` типа `str` содержит название навыка; валидатор `non_empty_name` запрещает пустые значения.

Поле `level` типа `Optional[Literal['junior', 'middle', 'senior']]` фиксирует уровень владения навыком; допускает `None`.

Поле `category` типа `Optional[str]` определяет категорию навыка, валидатор `strip_category` обрезает пробелы.

3.2.3 Класс `EducationEntry` описывает один период обучения кандидата.

Поле `institution` типа `str` хранит название учебного заведения и не может быть пустым.

Поле `degree` типа `Optional[str]` хранит степень или уровень образования.

Поле `field` типа `Optional[str]` хранит специальность.

Поле `graduation_year` типа `Optional[int]` хранит год окончания.

3.2.4 Класс `CandidateProfile` является центральной моделью домена. Он наследуется от `BaseModel` и содержит полную структурированную информацию о кандидате.

Поле `id` типа `UUID` является уникальным идентификатором кандидата; генерируется автоматически через `uuid4()` при отсутствии явного значения.

Поле `full_name` типа `Optional[str]` хранит полное имя кандидата.

Поле `email` типа `Optional[str]` хранит адрес электронной почты; валидатор `validate_email` приводит значение к нижнему регистру и проверяет формат регулярным выражением.

Поле `phone` типа `Optional[str]` хранит номер телефона; валидатор `normalize_phone` удаляет пробелы, тире и скобки, после чего проверяет, что остаток содержит от 7 до 15 цифр с необязательным префиксом `+`.

Поле `location` типа `Optional[str]` хранит город или регион кандидата.

Поле `languages` типа `list[str]` хранит список языков; валидатор `strip_languages` отфильтровывает пустые элементы и обрезает пробелы.

Поле `experience` типа `list[ExperienceEntry]` хранит список записей об опыте работы.

Поле `skills` типа `list[SkillEntry]` хранит список навыков.

Поле `education` типа `list[EducationEntry]` хранит список записей об образовании.

Поле `salary_expectation` типа `Optional[int]` хранит ожидаемую зарплату; валидатор `validate_salary` запрещает отрицательные значения.

Поле `years_of_experience` типа `Optional[float]` хранит суммарный стаж в годах; валидатор `validate_years` запрещает отрицательные значения.

Поле `raw_text_hash` типа `str` хранит SHA-256 хеш исходного текста резюме в виде строки из 64 шестнадцатеричных символов; валидатор `validate_hash` проверяет соответствие шаблону.

3.2.5 Классы `SuccessWrapper[T]` и `ErrorWrapper` (`libs/common/models/wrappers.py`) являются generic-обёртками ответов API. Класс `SuccessWrapper` является обобщённым и наследуется от `BaseModel` и `Generic[T]`.

Поле `success` имеет тип `Literal[True]` со значением по умолчанию `True`.

Поле `data` типа `T` содержит полезную нагрузку ответа.

Метод класса `of(data: T)` является фабричным методом, создающим экземпляр обёртки из данных.

Класс `ErrorWrapper` наследуется от `BaseModel`. Поле `success` имеет тип `Literal[False]` со значением по умолчанию `False`.

Поле `error` типа `str` содержит краткое описание ошибки.

Поле `detail` типа `Optional[str]` содержит расширенное описание.

Метод класса `of(error, detail)` является фабричным методом для создания ошибочного ответа.

3.2.6 Класс `ServiceException` (`libs/common/exceptions/base.py`) является базовым исключением для всех доменных ошибок.

Поле `status_code` класса имеет значение 500 по умолчанию.

Конструктор принимает `message` и опциональный `status_code`, переопределяющий значение класса. Метод `__repr__` возвращает строку с кодом статуса и сообщением.

3.3 Описание сервиса `extraction_service`

Данный сервис объединяет этапы загрузки документов, извлечения текста, обогащения метаданными и генерации эмбеддингов. Он является входным шлюзом системы и обеспечивает переход от неструктурированных файлов к векторным представлениям.

3.3.1 Класс `AppConfig` (`core/config.py`) наследуется от `BaseSettings` и является singleton-объектом конфигурации сервиса. Экземпляр `settings` создается при импорте модуля и используется всеми остальными компонентами.

Поле `log_level` типа `str` определяет уровень логирования; по умолчанию – `'INFO'`.

Поле `s3_endpoint` типа `str` задает URL-адрес S3-совместимого хранилища.

Поле `s3_bucket` типа `str` задает имя бакета; по умолчанию – `'resumes'`.

Поле `s3_access_key` и `s3_secret_key` типа `str` содержат учетные данные для доступа к S3.

Поле `embedding_api_url` типа `str` задает URL embedding API.

Поле `embedding_model` типа `str` задает имя модели для генерации эмбеддингов; по умолчанию – `'text-embedding-3-small'`.

Поле `embedding_batch_size` типа `int` определяет размер батча при пакетной генерации; по умолчанию 32.

Поле `llm_api_url` и `llm_model` типа `str` задают URL и имя языковой модели для извлечения сущностей.

Поле `ocr_engine` типа `str` указывает основной OCR-движок; по умолчанию – `'tesseract'`.

Поле `max_file_size_mb` типа `int` задает максимальный размер загружаемого файла в мегабайтах; по умолчанию – 20.

Поле `chunk_size` типа `int` определяет длину чанка в символах; по умолчанию – 512.

Поле `chunk_overlap` типа `int` определяет перекрытие между чанками; по умолчанию – 64.

Поле `ocr_llm_model` типа `str` задает имя vision-модели для OCR сложных изображений; пустая строка отключает LLM-режим.

Поле `ocr_confidence_threshold` типа `float` задает порог средней уверенности Tesseract, ниже которого подключается LLM; по умолчанию – 60.0.

Поле `ocr_llm_timeout` типа `float` задает таймаут запроса к vision-API в секундах; по умолчанию – 60.0.

Поле `ocr_min_chars_per_kb` типа `float` задает минимальную плотность символов (символов на КБ), ниже которой подключается LLM; по умолчанию – 3.0.

3.3.2 Класс `DocumentBlock` (`models/document.py`) наследуется от `BaseModel` и представляет один семантический блок внутри страницы документа.

Поле `block_type` типа `str` хранит тип блока согласно разметке Docling (например, 'text', 'title', 'table').

Поле `text` типа `str` содержит текстовое содержимое блока.

Поле `page` типа `int` указывает номер страницы, на которой расположен блок.

Поле `bbox` типа `Optional[list[float]]` содержит координаты ограничивающего прямоугольника блока в формате `[left, top, right, bottom]`; допускает `None`, если координаты недоступны.

3.3.3 Класс `DocumentPage` (`models/document.py`) наследуется от `BaseModel` и представляет одну страницу документа. Использование отдельного класса для страницы, а не плоского списка блоков с атрибутом `page`, упрощает итерацию по страницам при построении контекстного окна: можно ограничить чанк страницей, не выполняя группировку в runtime.

Поле `page_number` типа `int` хранит номер страницы.

Поле `blocks` типа `list[DocumentBlock]` содержит список семантических блоков страницы. Группировка блоков по страницам через `DocumentPage` сохраняет пространственную структуру документа, что позволяет при необходимости восстановить порядок разделов резюме и использовать номер страницы как дополнительный атрибут метаданных при индексации.

3.3.4 Класс `StructuredDocument` (`models/document.py`) наследуется от `BaseModel` и является сводным результатом обработки резюме, объединяющим все этапы пайплайна. Этот класс является центральным объектом передачи данных внутри `extraction-service`: он проходит через парсер, извлекатель сущностей, нарезчик чанков и генератор эмбедингов, последовательно обогащаясь данными. По завершении пайплайна `StructuredDocument` передается оркестратору, который разбирает его поля и сохраняет результаты в `storage-service`.

Поле `source_s3_key` типа `str` хранит ключ исходного файла в S3. Поле `full_text` типа `str` содержит полный извлеченный текст резюме.

Поле `pages` типа `list[DocumentPage]` хранит постраничную структуру документа.

Поле `candidate_profile` типа `Optional[CandidateProfile]` содержит структурированный профиль кандидата, извлеченный LLM; допускает `None` при ошибке извлечения.

Поле `chunks` типа `list[str]` содержит список текстовых фрагментов, полученных нарезкой `full_text`.

Поле `embeddings` типа `list[list[float]]` содержит векторные представления, соответствующие каждому чанку.

Поле `metadata` типа `dict` хранит вспомогательные метаданные, такие как имя парсера и исходное имя файла.

3.3.5 Модуль `s3_service` (`services/s3_service.py`) реализует слой взаимодействия с S3-совместимым объектным хранилищем через библиотеку `aioboto3`. Модуль использует единственную разделяемую сессию `_session = aioboto3.Session()` и внутреннюю функцию `_client()`, создающую контекстный клиент с параметрами из `settings`.

Функция `async upload_file(file_bytes: bytes, filename: str) => str` генерирует уникальный S3-ключ вида `<uuid>.<ext>`, загружает байты файла в бакет через `put_object` и возвращает ключ. После успешной загрузки фиксирует событие `s3.uploaded` с ключом и размером файла.

Функция `async download_file(s3_key: str) => bytes` скачивает объект из бакета по ключу через `get_object`, читает тело ответа и возвращает байты. Фиксирует событие `s3.downloaded` с ключом и размером. Использование единственной сессии `_session` вместо создания новой сессии при каждом вызове снижает накладные расходы: `aioboto3.Session` является легковесным объектом, но его создание предполагает инициализацию ряда внутренних структур; совместное использование одной сессии также позволяет повторно применять пул соединений при высоком темпе загрузок.

3.3.6 Класс `_OcrMetrics` (`services/parsers/image_parser.py`) является именованным кортежем (`NamedTuple`) и содержит результаты первичного OCR-прохода Tesseract.

Поле `text` типа `str` содержит распознанный текст.

Поле `mean_conf` типа `float` содержит среднюю уверенность Tesseract.

Поле `char_density` типа `float` содержит плотность символов – отношение количества символов к размеру файла в килобайтах.

Функция `_ocr_sync_with_metrics(file_bytes: bytes) => _OcrMetrics` выполняется синхронно в пуле потоков. Она открывает изображение через Pillow, преобразует его в режим RGB при необходимости, вызывает `pytesseract.image_to_data` для получения уверенностей по каждому слову, затем `pytesseract.image_to_string` для получения полного текста. Возвращает `_OcrMetrics` с текстом, средней уверенностью и плотностью символов.

Функция `_pixel_entropy(file_bytes: bytes) => float` вычисляет информационную энтропию Шеннона по гистограмме пикселей в оттенках серого. Значение ниже 2.0 указывает на пустое или монохромное изображение, значение от два до пять – на ограниченный контраст, от пять до восемь – на полноценное содержимое.

Функция `_assess_quality(metrics, entropy, conf_threshold, density_threshold) => tuple[bool, str]` принимает метрики OCR и пороговые значения из конфигурации и возвращает пару: флаг необходимости LLM и читаемое описание причины. Возвращает `True` если энтропия ниже 2.0, средняя уверенность ниже порога или плотность символов ниже порога.

Функция `async _llm_vision_ocr(file_bytes: bytes, filename: str) => str` кодирует изображение в base64, формирует data-URL с MIME-типом и отправляет запрос к OpenAI-совместимому chat/completions API с системным промптом для распознавания резюме. Возвращает текст из первого варианта ответа.

Публичная функция `async parse_image(file_bytes: bytes, filename: str) => str` является входной точкой OCR-пайплайна. Выполняет валидацию расширения, запускает `_ocr_sync_with_metrics` и `_pixel_entropy` в пуле потоков, оценивает качество через `_assess_quality`. Если качество низкое и `ocr_llm_model` задан – вызывает `_llm_vision_ocr`. Возвращает более длинный из двух результатов (Tesseract или LLM).

3.3.7 Модуль `docling_parser` (`services/parsers/docling_parser.py`) реализует разбор PDF и DOCX файлов с помощью библиотеки Docling. Использует ленивый singleton-конвертер `_converter`, который инициализируется при первом обращении через функцию `_get_converter()`. Класс `ParseError` наследуется от `Exception` и выбрасывается при неудаче парсинга.

Функция `_build_converter() => DocumentConverter` создает конвертер Docling с включенным OCR для PDF через `PdfPipelineOptions(do_ocr=True)`.

Функция `_convert_sync(tmp_path: str) => DoclingDocument` вызывает `_get_converter().convert(tmp_path)` синхронно – она передается в `run_in_executor` для выполнения в пуле потоков.

Функция `_docling_to_structured(doc: DoclingDocument, s3_key: str) => StructuredDocument` обходит все элементы документа через `doc.iterate_items()`, извлекает текст, номер страницы и координаты bounding box, формирует объекты `DocumentBlock` и группирует их по страницам в `DocumentPage`. Возвращает `StructuredDocument` с полным текстом, постраничной структурой и метаданными парсера.

Публичная функция `async parse(file_bytes: bytes, filename: str, s3_key: str) => StructuredDocument` записывает байты во временный файл, запускает `_convert_sync` и удаляет временный файл.

3.3.8 Модуль `entity_extractor` (`services/parsers/entity_extractor.py`) вызывает LLM для извлечения структурированного профиля кандидата из текста резюме.

Вспомогательные внутренние классы `_Message`, `_ResponseFormat`, `_ChatRequest` являются Pydantic-моделями для формирования тела запроса к

chat/completions API: `_ChatRequest` содержит поля `model`, `messages: list[_Message]`, `temperature: float` со значением `0.0` и `response_format: _ResponseFormat` с типом `json_object` для получения гарантированного JSON.

Публичная функция `async extract(full_text: str) => CandidateProfile` форматирует промпт `ENTITY_EXTRACTION_PROMPT` с текстом резюме, отправляет POST-запрос на `llm_api_url/chat/completions`, парсит JSON из ответа и валидирует его через `CandidateProfile.model_validate`. При ошибке декодирования JSON выбрасывает `ValueError`. Успешное извлечение логируется с именем кандидата.

3.3.9 Модуль `embedding_service` (`services/embedding_service.py`) реализует пакетную генерацию векторных эмбеддингов.

Внутренние классы `_EmbedRequest` (`model: str, input: list[str]`), `_EmbeddingObject` (`embedding: list[float], index: int`) и `_EmbedResponse` (`data: list[_EmbeddingObject]`) формируют типизированный интерфейс к `embedding` API.

Функция `async _embed_single_batch(client, texts) => list[list[float]]` отправляет один батч текстов на `/embeddings`, десериализует ответ через `_EmbedResponse`, сортирует результаты по полю `index` и возвращает упорядоченный список векторов.

Публичная функция `async embed_batch(texts: list[str]) => list[list[float]]` нарезает входной список на батчи размером `embedding_batch_size`, последовательно вызывает `_embed_single_batch` для каждого батча в рамках одного `httpx.AsyncClient` и возвращает объединенный список векторов. При пустом входе возвращает пустой список.

3.3.10 Модуль `extraction_service` (`services/extraction_service.py`) реализует главный пайплайн обработки резюме.

Функция `_chunk_text(text: str, chunk_size: int, overlap: int) => list[str]` разбивает текст на фрагменты фиксированного размера с перекрытием: каждый следующий чанк начинается со смещением `chunk_size - overlap` символов. Перекрытие обеспечивает сохранность семантического контекста на границах чанков, что повышает точность векторного поиска по фрагментарным запросам.

Публичная функция `async process_resume(file_bytes: bytes, filename: str, s3_key: str) => StructuredDocument` является оркестратором пайплайна. Определяет формат файла по расширению: для изображений вызывает `parse_image`, для PDF/DOCX – `docling_parser.parse` с откатом на `raw_parsers` при ошибке. Затем последовательно вызывает `extract_entities` для построения профиля, `_chunk_text` для нарезки текста и `embed_batch` для получения эмбеддингов. Каждый шаг обернут в `try/except` – при ошибке извлечения или эмбеддинга

соответствующее поле устанавливается в `None` или пустой список. Возвращает `StructuredDocument` с заполненными полями и метаданными. Выбор откатного парсера осуществляется в порядке приоритета: сначала `pdfplumber` для PDF, затем `python-docx` для DOCX; результат парсинга всегда приводится к единому формату `StructuredDocument` независимо от использованного парсера.

3.3.11 Модуль `text_processing_service` (`services/text_processing_service.py`) является высокоуровневым фасадом, связывающим скачивание из S3 и запуск пайплайна.

Функция `async process_document_by_key(s3_key: str, filename: str) => StructuredDocument` скачивает файл из S3 по ключу через `s3_service.download_file`, затем передает байты в `process_resume`. Логирует размер скачанного файла, количество чанков и эмбедингов, наличие профиля.

Функция `async reprocess_document(s3_key: str, filename: str) => StructuredDocument` является именованным алиасом для `process_document_by_key`, предназначенным для читаемости кода в точках повторной обработки.

3.3.12 Модуль `router` (`routers/extraction_router.py`) определяет HTTP-эндпоинты сервиса `extraction-service`. Все маршруты декорированы аутентификационной зависимостью `require_auth` и возвращают ответы, обернутые в `SuccessWrapper[T]` или `ErrorWrapper`.

Маршрут `POST /extract/upload` принимает `multipart`-форму с полем `file` типа `UploadFile` и необязательным `correlation_id: str`. Валидирует расширение и размер файла (не более `settings.max_file_size_mb` МБ). Читает байты файла, генерирует S3-ключ через `s3_service.upload_file`, затем запускает `process_resume` в фоне через `BackgroundTasks`. Немедленно возвращает `SuccessWrapper` с `{ "s3_key": key, "correlation_id": cid, "status": "processing" }`.

Маршрут `POST /extract/process` принимает JSON-тело с полем `s3_key: str` и `filename: str`. Вызывает `text_processing_service.process_document_by_key` синхронно (ожидает результат) и возвращает `SuccessWrapper[StructuredDocument]`. Предназначен для использования оркестратором при синхронной обработке.

Маршрут `GET /extract/health` возвращает `{ "status": "ok", "service": "extraction-service" }` без аутентификации – используется `Docker healthcheck` и оркестратором.

Класс `ExtractionUploadResponse` (`models/response.py`) наследуется от `BaseModel` и содержит поля `s3_key: str`, `correlation_id: str` и `status: str`. Используется в качестве типа `T` в `SuccessWrapper[ExtractionUploadResponse]` при немедленном ответе на загрузку. Разделение маршрутов `/extract/upload` (асинхронный) и

`/extract/process` (синхронный) позволяет BFF выбирать стратегию вызова в зависимости от контекста: загрузка через пользовательский интерфейс использует асинхронный маршрут с WebSocket-оповещением, а оркестратор использует синхронный маршрут для последовательного выполнения агентских шагов.

3.4 Описание сервиса `storage_service`

Данный сервис обеспечивает долговременное хранение векторных представлений профилей кандидатов вместе с ассоциированными метаданными. Он отвечает за эффективный доступ к накопленным данным, поддержку приближённого поиска по сходству, фильтрацию по структурированным атрибутам и инкрементальное обновление коллекций. Сервис построен вокруг векторной базы Qdrant и полностью соответствует блоку векторного хранилища и индексации дипломного проекта. Взаимодействие с Qdrant осуществляется исключительно через асинхронный клиент `AsyncQdrantClient`, что исключает блокировку `event loop` при длительных операциях пакетной загрузки.

3.4.1 Класс `CandidateVectorPayload` (`models/vector.py`) наследуется от `BaseModel` и представляет метаданные, сохраняемые вместе с вектором в Qdrant.

Поле `candidate_id` типа `UUID` является идентификатором кандидата.

Поле `full_name` типа `str | None` хранит полное имя.

Поле `skills` типа `list[str]` хранит список навыков в нижнем регистре для фильтрации.

Поле `location` типа `str | None` хранит местоположение.

Поле `years_of_experience` типа `float | None` хранит суммарный стаж в годах.

Поле `salary_expectation` типа `int | None` хранит ожидаемую зарплату.

Поле `languages` типа `list[str]` хранит список языков.

Поле `source_s3_key` типа `str` хранит ключ исходного файла в S3.

Поле `chunk_index` типа `int` хранит индекс чанка, которому соответствует данная точка.

Поле `chunk_text` типа `str` хранит текстовое содержимое чанка.

Поле `indexed_at` типа `datetime` хранит временную метку индексации.

Поле `version` типа `int` хранит версию записи; по умолчанию – 1.

3.4.2 Класс `EmbeddingPoint` (`models/vector.py`) наследуется от `BaseModel` и представляет полную точку Qdrant, готовую к загрузке.

Поле `id` типа `UUID` является уникальным идентификатором точки.

Поле `vector` типа `list[float]` содержит векторное представление чанка.

Поле `payload` типа `CandidateVectorPayload` содержит метаданные точки.

3.4.3 Класс `QdrantService` (`services/qdrant_service.py`) является оберткой над `AsyncQdrantClient` из библиотеки `qdrant-client`.

Конструктор инициализирует поле `_client` экземпляром `AsyncQdrantClient(url=settings.qdrant_url)` и поле `_collection` именем коллекции из конфигурации.

Метод `async upsert(point: EmbeddingPoint) => str` преобразует объект `EmbeddingPoint` в структуру `PointStruct`, вызывает `_client.upsert` с единственной точкой и возвращает строковое представление идентификатора точки. Фиксирует событие `qdrant.upsert` в журнале.

Метод `async batch_upsert(points: list[EmbeddingPoint]) => list[str]` преобразует весь список точек в `list[PointStruct]` и вызывает `_client.upsert` одним вызовом для эффективной пакетной загрузки. Возвращает список идентификаторов всех загруженных точек. Фиксирует событие `qdrant.batch_upsert` с количеством точек.

Метод `async delete(point_id: str) => None` удаляет точку по идентификатору через `_client.delete` с `PointIdsList`. Фиксирует событие `qdrant.delete`.

Метод `async get_by_id(point_id: str) => EmbeddingPoint | None` извлекает точку по идентификатору через `_client.retrieve` с флагами `with_vectors=True` и `with_payload=True`. При отсутствии результата возвращает `None`, при успехе восстанавливает объект `EmbeddingPoint` из записи. Передача флага `with_vectors=True` необходима для получения полного вектора точки, который может потребоваться при операциях ре-индексации; при использовании метода исключительно для отображения профиля кандидата флаг допустимо установить в `False` для экономии трафика.

3.4.4 Класс `CollectionService` (`services/collection_service.py`) управляет жизненным циклом Qdrant-коллекции.

Конструктор инициализирует поля `_client` и `_collection` аналогично `QdrantService`. Разделение обязанностей между `QdrantService` (CRUD-операции над точками) и `CollectionService` (управление схемой коллекции) следует принципу единственной ответственности: изменение конфигурации индексов не затрагивает код работы с точками.

Метод `async ensure_collection() => None` проверяет существование коллекции через `_client.get_collections()`. Если коллекция отсутствует – создает ее с параметрами: метрика расстояния `COSINE`, размерность вектора из `settings.vector_size`, конфигурация `HNSW` с `m=16`, `ef_construct=100`, `full_scan_threshold=10000`. После создания коллекции создает `payload-индексы`: тип `KEYWORD` для полей `skills`, `location`, `languages`; тип `FLOAT` для `years_of_experience`; тип `INTEGER` для `salary_expectation`. При

существующей коллекции логирует `collection.already_exists` и возвращает управление.

Метод `async list_collections() => list[dict]` возвращает список словарей с полем `name` для каждой коллекции в Qdrant.

3.4.5 Класс `AppConfig` (`core/config.py` сервиса `storage-service`) наследуется от `BaseSettings` и является singleton-конфигурацией сервиса хранения.

Поле `log_level` типа `str` определяет уровень логирования; по умолчанию – `'INFO'`.

Поле `qdrant_url` типа `str` задает URL-адрес Qdrant; по умолчанию – `'http://localhost:6333'`.

Поле `qdrant_collection` типа `str` задает имя коллекции; по умолчанию – `'candidates'`.

Поле `vector_size` типа `int` задает размерность векторов; по умолчанию – 1536.

Поле `auth_service_url` типа `str` задает URL `auth-service` для валидации JWT.

Поле `s3_endpoint`, `s3_bucket`, `s3_access_key` и `s3_secret_key` типа `str` задают параметры S3-совместимого хранилища, аналогично `extraction-service`.

3.4.6 Модуль `router` (`routers/storage_router.py`) определяет HTTP-эндпоинты сервиса `storage-service`. Все маршруты, кроме `/storage/init` и `/storage/collections`, требуют валидного JWT в заголовке `Authorization`; зависимость `require_auth` верифицирует токен через `auth-service`. Разделение операций записи (`upsert`, `batch`) и чтения (`GET`) на отдельные маршруты упрощает реализацию ролевого доступа: запись разрешена только ролям `system` и `admin`, тогда как чтение доступно всем аутентифицированным пользователям.

Маршрут `POST /storage/upsert` принимает JSON-тело с полем `point: EmbeddingPoint`. Вызывает `qdrant_service.upsert(point)` и возвращает `SuccessWrapper[str]` с идентификатором сохраненной точки.

Маршрут `POST /storage/batch` принимает JSON-тело с полем `points: list[EmbeddingPoint]`. Вызывает `qdrant_service.batch_upsert(points)` и возвращает `SuccessWrapper[list[str]]` со списком идентификаторов. Пакетный маршрут предназначен для первоначальной загрузки большого числа резюме: единственный вызов `_client.upsert` со списком точек эффективнее последовательных одиночных вызовов, поскольку Qdrant оптимизирует запись в HNSW-граф при получении нескольких точек за одну операцию.

Маршрут `DELETE /storage/{point_id}` принимает идентификатор точки из пути и вызывает `qdrant_service.delete(point_id)`. Возвращает `SuccessWrapper[dict]` с подтверждением удаления `{"deleted":`

`point_id`}. Операция является идемпотентной: повторный вызов для уже удаленной точки не вызывает ошибки – Qdrant возвращает статус `ok` без исключения.

Маршрут `GET /storage/{point_id}` принимает идентификатор точки. Вызывает `qdrant_service.get_by_id(point_id)`: при результате `None` выбрасывает `NotFoundError`, при успехе возвращает `SuccessWrapper[EmbeddingPoint]`. Маршрут предназначен для получения полного профиля кандидата из BFF.

Маршрут `GET /storage/collections` вызывает `collection_service.list_collections()` и возвращает `SuccessWrapper[list[dict]]`. Предназначен для мониторинга и административных задач.

Маршрут `POST /storage/init` вызывает `collection_service.ensure_collection()` и возвращает `SuccessWrapper[dict]` с сообщением о статусе инициализации коллекции. Вызывается при старте сервиса через `lifespan`-обработчик.

3.5 Описание сервиса `search_service`

Модуль `search-service` реализует блок гибридного поиска, объединяющий векторный поиск, ранжирование, нейронный реранкинг и бизнес-правила для итоговой оценки кандидатов. `CrossEncoder` применяется только к предварительно отфильтрованному подмножеству кандидатов, а не ко всей базе данных.

3.5.1 Класс `MetadataFilters` (`models/request.py`) наследуется от `BaseModel` и содержит параметры фильтрации поиска.

Поле `skills_required` типа `list[str]` задает обязательные навыки; кандидат должен владеть всеми указанными навыками (AND-семантика).

Поле `skills_preferred` типа `list[str]` задает желательные навыки; достаточно наличия хотя бы одного (OR-семантика).

Поле `location` типа `str | None` задает точное совпадение по местоположению.

Поле `min_experience_years` типа `float | None` задает минимальный стаж работы.

Поле `max_salary` типа `int | None` задает верхний порог зарплатных ожиданий кандидата.

Поле `languages` типа `list[str]` задает желаемые языки; OR-семантика.

3.5.2 Класс `SearchRequest` (`models/request.py`) наследуется от `BaseModel` и является входным объектом поискового запроса.

Поле `query` типа `str` содержит текст поискового запроса на естественном языке.

Поле `filters` типа `MetadataFilters` содержит параметры фильтрации; по умолчанию – пустой объект без ограничений.

Поле `top_k` типа `int` определяет количество возвращаемых кандидатов; по умолчанию — 10.

Поле `use_rerank` типа `bool` управляет включением нейронного ре-ранкинга; по умолчанию — `True`.

Поле `correlation_id` типа `str` передает идентификатор запроса для трассировки.

3.5.3 Класс `RankedCandidate` (`models/response.py`) наследуется от `BaseModel` и представляет одного кандидата в ответе на поисковый запрос.

Поле `candidate_id` типа `UUID` является идентификатором кандидата.

Поле `full_name` типа `str | None` содержит имя кандидата.

Поле `vector_score` типа `float` содержит нормализованную оценку семантической близости.

Поле `rerank_score` типа `float` содержит нормализованную оценку `CrossEncoder`.

Поле `final_score` типа `float` содержит итоговую взвешенную оценку с учетом бизнес-бонусов.

Поле `matched_skills` типа `list[str]` содержит пересечение навыков кандидата с требуемыми.

Поле `explanation` типа `str` содержит человекочитаемое объяснение оценки в виде строки с компонентами.

Поле `chunk_text` типа `str` содержит текстовое резюме ключевых атрибутов кандидата.

3.5.4 Класс `SearchResponse` (`models/response.py`) содержит поле `total` типа `int` с общим числом найденных кандидатов и поле `candidates` типа `list[RankedCandidate]` с отсортированным списком результатов.

3.5.5 Класс `_BM25` (`services/fulltext_filter.py`) реализует алгоритм ранжирования BM25 без внешних зависимостей. Константы `_K1 = 1.5` и `_B = 0.75` соответствуют стандартным.

Конструктор принимает `corpus: list[list[str]]` — список токенизированных документов. Вычисляет число документов, среднюю длину документа, частоту каждого термина в каждом документе и частоту документов для каждого термина.

Метод `score(query_tokens: list[str], doc_index: int) => float` вычисляет оценку для одного документа.

Функция `_tokenize(text: str) => list[str]` приводит текст к нижнему регистру и разбивает по нелитерным символам.

Публичная функция `bm25_rerank(query: str, payloads: list[dict]) => list[int]` принимает список `payload`-словарей из `Qdrant`, токенизирует их поле `chunk_text`, строит объект `_BM25` и возвращает список индексов, отсортированных по убыванию оценки.

Публичная функция `bm25_scores(query: str, payloads: list[dict]) => list[float]` возвращает BM25-оценки в исходном порядке документов. Разделение на `bm25_rerank` (возвращает индексы) и `bm25_scores` (возвращает оценки) позволяет вызывающему коду выбирать нужный формат: `bm25_rerank` удобен при перестановке списка, `bm25_scores` — при покомпонентном вычислении финальной оценки в `search_service`.

3.5.6 Модуль `qdrant_search` (`services/qdrant_search.py`) реализует векторный поиск с построением Qdrant-фильтра.

Глобальная переменная `_client` хранит lazy-инициализированный экземпляр `AsyncQdrantClient`; функция `get_client()` реализует паттерн `singleton`.

Функция `_build_filter(filters: MetadataFilters) => Filter | None` формирует объект фильтра Qdrant: для `skills_required` создает список `FieldCondition` с `MatchValue` (AND-семантика через поле `must`); для `skills_preferred` и `languages` — `FieldCondition` с `MatchAny` (OR-семантика через поле `should`); для `location` — точное совпадение; для `min_experience_years` — `Range(gte=...)`, для `max_salary` — `Range(lte=...)`. Возвращает `None` если фильтры не заданы.

Функция `async vector_search(vector, filters, top_n) => list[ScoredPoint]` вызывает `_client.search` с вектором запроса, построенным фильтром, лимитом `top_n`, флагами `with_payload=True` и `with_vectors=False`.

3.5.7 Модуль `reranker` (`services/reranker.py`) реализует нейронный ре-ранкинг на основе `CrossEncoder` из библиотеки `sentence-transformers`.

Функция `_get_model() => CrossEncoder` загружает модель из `settings.reranker_model` с кешированием через декоратор `@lru_cache(maxsize=1)`, обеспечивая однократную инициализацию.

Функция `_candidate_to_text(candidate: dict) => str` синтезирует текстовое представление кандидата для передачи в `CrossEncoder`: объединяет имя, местоположение, стаж, навыки, языки и опыт работы через разделитель.

Функция `_predict(query, candidates) => list[float]` формирует пары `[query, candidate_text]` и вызывает `model.predict` синхронно.

Публичная функция `async rerank(query: str, candidates: list[dict]) => list[RerankScore]` запускает `_predict`. Возвращает список объектов `RerankScore` с полями `index` и `score` для каждого кандидата.

3.5.8 Модуль `search_service` (`services/search_service.py`) реализует главный поисковый пайплайн.

Функция `async _embed_query(query: str) => list[float]` отправляет текст запроса на `embedding API` и возвращает первый вектор из ответа.

Функция `_minmax(values: list[float]) => list[float]` нормализует список оценок в диапазон `[0, 1]` по формуле min-max. При равных значениях возвращает список из единиц.

Функция `_business_rules_score(payload: dict, filters: MetadataFilters) => float` вычисляет бонус за совпадение предпочтительных навыков (по `_PREFERRED_SKILL_BONUS = 0.02` за каждый) и языков (`_LANGUAGE_BONUS = 0.03`). Итоговый бонус ограничен константой `_BUSINESS_BONUS_CAP = 0.10`.

Функция `_matched_skills(payload, filters) => list[str]` возвращает отсортированное пересечение навыков кандидата с объединением `skills_required` и `skills_preferred`. Навыки сравниваются в нижнем регистре, что устраняет расхождения между написаниями «Python» и «python» при поступлении данных из разных источников.

Функция `_explanation(payload, filters, vector_score, rerank_score, business_bonus) => str` формирует строку объяснения вида `"vector=0.812; rerank=0.743; bonus=0.040; skills: python, fastapi"`.

Публичная функция `async run_search(req: SearchRequest) => SearchResponse` выполняет шесть последовательных шагов: один) получает вектор запроса через `_embed_query`; два) выполняет векторный поиск в Qdrant с `prefetch_n = top_k * prefetch_multiplier` результатами; три) при `use_rerank=True` вызывает `reranker.rerank`, иначе заполняет нулевыми оценками; четыре) нормализует векторные и ре-ранк оценки через `_minmax`; пять) вычисляет `final_score = vector_weight * v_score + rerank_weight * r_score + bonus` для каждого кандидата; шесть) сортирует по убыванию `final_score`, обрезает до `top_k` и возвращает `SearchResponse`.

3.5.9 Класс AppConfig (core/config.py сервиса search-service) наследуется от `BaseSettings` и является singleton-конфигурацией поискового сервиса.

Поле `log_level` типа `str` определяет уровень логирования; по умолчанию – `'INFO'`.

Поле `qdrant_url` типа `str` задает URL Qdrant; по умолчанию – `'http://localhost:6333'`.

Поле `qdrant_collection` типа `str` задает имя коллекции для поиска.

Поле `embedding_api_url` и `embedding_model` типа `str` задают URL и модель embedding API для векторизации запросов пользователя.

Поле `reranker_model` типа `str` задает имя модели CrossEncoder из `sentence-transformers`.

Поле `prefetch_multiplier` типа `int` определяет коэффициент предварительной выборки из Qdrant перед ре-ранкингом.

Поле `vector_weight` типа `float` определяет вес векторной оценки в финальной формуле; по умолчанию – `0.4`.

Поле `rerank_weight` типа `float` определяет вес ре-ранк оценки; по умолчанию — 0.6.

Поле `auth_service_url` типа `str` задает URL auth-service для валидации JWT.

3.5.10 Модуль `router` (`routers/search_router.py`) определяет HTTP-эндпоинты сервиса `search-service`.

Класс `RerankScore` (`models/response.py`) наследуется от `BaseModel` и является вспомогательной моделью результата ре-ранкинга. Поле `index` типа `int` хранит позицию кандидата в исходном списке. Поле `score` типа `float` хранит оценку `CrossEncoder`.

Маршрут `POST /search` принимает JSON-тело типа `SearchRequest` с обязательной аутентификацией. Вызывает `search_service.run_search(req)` и возвращает `SuccessWrapper[SearchResponse]` с отсортированным списком кандидатов. Зависимость `require_auth` проверяет JWT через `AuthClient.validate_token`; при невалидном токене выбрасывается `UnauthorizedError` до выполнения поискового запроса.

Маршрут `POST /search/rerank` принимает JSON-тело с полями `query: str` и `candidates: list[dict]`. Вызывает `reranker.rerank(query, candidates)` напрямую, минуя векторный поиск, и возвращает `SuccessWrapper[list[RerankScore]]`. Предназначен для использования оркестратором при необходимости изолированного ре-ранкинга. Отдельный эндпоинт для ре-ранкинга позволяет агенту выполнить дополнительную переоценку результатов после их обогащения данными из `web_search` или `get_job_description`, не повторяя дорогостоящий векторный поиск в `Qdrant`.

Маршрут `GET /search/health` возвращает статус сервиса и состояние подключения к `Qdrant` без аутентификации.

3.6 Классы модуля `orchestrator-service`

Модуль `orchestrator-service` (порт 8004) является центральным блоком оркестрации агентов на основе `LangGraph`. Он координирует работу всех остальных сервисов через агентов с инструментами и управляет состоянием задач.

3.6.1 Класс `OrchestratorState` (`models/state.py`) наследуется от `BaseModel` и является схемой состояния графов `LangGraph`.

Поле `user_query` типа `str` хранит исходный запрос пользователя.

Поле `query_type` типа `Literal['upload', 'search', 'batch', 'get_profile']` определяет тип задачи; по умолчанию — 'search'.

Поле `correlation_id` типа `str` является идентификатором запроса для трассировки.

Поле `auth_token` типа `str` хранит JWT-токен для авторизации вызовов к другим сервисам.

Поле `extracted_document` типа `Optional[dict]` хранит результат извлечения данных из резюме.

Поле `candidate_profile` типа `Optional[dict]` хранит профиль кандидата после сохранения.

Поле `search_results` типа `Optional[list[dict]]` хранит список результатов поиска.

Поле `reflection` типа `Optional[dict]` хранит оценку качества результатов от LLM.

Поле `retry_count` типа `int` хранит текущее число попыток; по умолчанию – 0.

Поле `revised_query` типа `Optional[str]` хранит скорректированный запрос, предложенный `reflection`-агентом.

Поле `max_retries` типа `int` определяет максимально допустимое число повторов; по умолчанию – 2.

Поле `messages` типа `list` хранит накопленные сообщения агентов `LangChain`.

Поле `error` типа `Optional[str]` хранит сообщение об ошибке при ее возникновении.

Поле `metadata` типа `dict` хранит вспомогательные данные (имя файла, байты, `point_id`).

3.6.2 Класс `OrchestratorService` (`services/orchestrator_service.py`) является `singleton`-объектом, управляющим жизненным циклом графов `LangGraph`.

Конструктор инициализирует три приватных поля: `_search_graph`, `_upload_graph` и `_checkpoint` – все со значением `None`.

Метод `async initialize() => None` создает `checkpoint` через `make_checkpoint()`, компилирует граф поиска через `build_search_graph(checkpointer=...)` и граф загрузки через `build_upload_graph(checkpointer=...)`. Логирует событие инициализации.

Метод `async shutdown() => None` закрывает все HTTP-клиенты сервисов (`extraction_client`, `storage_client`, `search_client`, `auth_client`) параллельно через `asyncio.gather` с флагом `return_exceptions=True`. Флаг `return_exceptions=True` предотвращает прерывание процесса завершения из-за ошибки в одном из клиентов: даже если закрытие одного соединения вызовет исключение, остальные клиенты будут закрыты корректно.

Метод `async run(state: OrchestratorState) => OrchestratorState` выбирает граф через внутренний метод `_select_graph` по полю `query_type` (типы `'search'` и `'get_profile'` маршрутизируются в `_search_graph`, типы `'upload'` и `'batch'` – в `_upload_graph`). Запускает

граф через `ainvoke` с таймаутом `settings.agent_timeout_sec`. При превышении таймаута выбрасывает `AgentTimeoutError`, при других исключениях – `OrchestratorError`. Метод является единственной публичной точкой входа для запуска агентских сценариев: все HTTP-эндпоинты оркестратора проксируют управление через него, обеспечивая централизованное управление таймаутами и обработкой ошибок.

Метод `async get_status(correlation_id: str) => dict | None` запрашивает последний снимок состояния из `_search_graph.get_state`, при отсутствии – из `_upload_graph.get_state`. Возвращает словарь с полями `correlation_id`, `values`, `next` (список следующих узлов) и `done` (флаг завершения).

3.6.3 Функция `build_search_graph(checkpointer=None)` (`graphs/candidate_search_graph.py`) строит и компилирует граф поиска `LangGraph`.

Граф содержит пять узлов: `router` выполняет классификацию запроса через `route_query`; `search` вызывает `search_agent` и извлекает результаты из потока сообщений через `_parse_search_results`; `reflect` вызывает `reflection_node` для оценки качества; `format` добавляет итоговые метаданные; `error` фиксирует ошибку через `error_handler`.

Ребра: `router => search`; условный переход после `search` – в `reflect` или `error`; условный переход после `reflect` через `should_retry` – в `search` (повтор) или `format` (завершение). Узлы `format` и `error` ведут в `END`.

3.6.4 Функция `build_upload_graph(checkpointer=None)` (`graphs/resume_upload_graph.py`) строит граф загрузки резюме.

Граф содержит четыре узла: `router` классифицирует запрос; `extract` запускает `extraction_agent`, передавая байты файла через `configurable`, и извлекает результат через `_parse_extracted_document`; `store` запускает `storage_agent` с профилем и эмбедингами, извлекает `point_id` через `_parse_point_id`; `error` обрабатывает ошибки.

Условный переход после `extract`: в `store` при наличии результата, в `error` при ошибке или пустом документе.

Условный переход после `store`: в `END` при успехе, в `error` при ошибке. Байты файла передаются через механизм `configurable` `LangGraph`, а не через поле состояния, поскольку `OrchestratorState` является Pydantic-моделью и не подходит для хранения крупных бинарных данных.

3.6.5 Модуль `common_nodes` (`graphs/common_nodes.py`) содержит разделяемые узлы обоих графов.

Функция `async reflection_node(state: OrchestratorState) => OrchestratorState` отправляет топ-3 результата поиска в LLM с промптом `REFLECTION_PROMPT` и парсит JSON-ответ с оценкой качества (`quality`) и

опционально скорректированным запросом (`revised_query`). При ошибке парсинга устанавливает `reflection = {"quality": "ok"}`.

Функция `should_retry(state: OrchestratorState) => Literal["retry", "done"]` является условным ребром: возвращает "retry" если `reflection.quality == "low"` и `retry_count < max_retries`, иначе — "done".

Функция `async error_handler(state: OrchestratorState) => OrchestratorState` фиксирует ошибку в логе, устанавливает `retry_count = max_retries` для предотвращения дальнейших повторов и `reflection = {"quality": "ok"}` для корректного выхода из условного ребра.

3.6.6 Функция `async route_query(user_query: str)` (`services/agents/router_agent.py`) классифицирует запрос пользователя через LLM со structured output. Использует класс `RouterOutput` (`BaseModel` с полем `query_type: Literal["upload", "search", "batch", "get_profile"]`) и вызывает `llm.with_structured_output(RouterOutput).ainvoke(prompt)`.

Возвращает значение `query_type`. Применение structured output вместо текстового парсинга исключает ошибки распознавания: LLM гарантированно возвращает одно из четырех допустимых значений, что предотвращает неопределенное поведение при нестандартных формулировках запроса.

3.6.7 Объект `search_agent` (`services/agents/search_agent.py`) создается функцией `build_search_agent()` через `create_react_agent` с моделью LLM и тремя инструментами: `hybrid_search`, `web_search`, `get_job_description`.

Системный промпт указывает агенту использовать `hybrid_search` несколько раз с разными формулировками при необходимости. Инструмент `web_search` применяется в случаях, когда в базе данных отсутствуют кандидаты с требуемыми навыками: агент может найти актуальные данные о технологиях и сформулировать альтернативные варианты запроса для повторного поиска.

Инструмент `get_job_description` позволяет агенту обогатить исходный запрос типовыми требованиями к должности, если рекрутер указал только название роли без перечня навыков.

3.6.8 Объект `extraction_agent` (`services/agents/extraction_agent.py`) создается через `create_react_agent` с инструментом `extract_resume`. Агент имеет единственный инструмент и минимальный системный промпт, поскольку его задача строго ограничена: принять файл, вызвать инструмент извлечения и вернуть структурированный результат. Упрощенная конфигурация снижает вероятность ошибочных действий LLM и сокращает задержку выполнения агентского цикла.

3.6.9 Объект `storage_agent` (`services/agents/storage_agent.py`) создается через `create_react_agent` с инструментом `store_candidate_profile`.

3.6.10 Инструмент `extract_resume` (`services/tools/extraction_tools.py`) декорирован `@tool` и принимает `file_bytes: bytes` и `correlation_id: str`. Вызывает `extraction_client.process_resume` и возвращает словарь с результатом обработки.

3.6.11 Инструмент `hybrid_search` (`services/tools/search_tools.py`) декорирован `@tool` и принимает `query: str`, `filters: dict` и `top_k: int`. Вызывает `search_client.hybrid_search` и возвращает список словарей с результатами.

3.6.12 Инструмент `web_search` декорирован `@tool` и принимает `query: str`. Выполняет POST-запрос на DuckDuckGo HTML-поиск (<https://html.duckduckgo.com/html/>), извлекает заголовки и сниппеты регулярными выражениями, очищает HTML-теги и возвращает до пять результатов в виде нумерованного текста.

3.6.13 Инструмент `store_candidate_profile` (`services/tools/storage_tools.py`) декорирован `@tool` и принимает `profile: dict` и `embeddings: list[float]`. Формирует структуру точки с идентификатором из профиля или новым UUID, вызывает `storage_client.upsert` и возвращает строку с `point_id`.

3.6.14 Инструмент `get_job_description` (`services/tools/jd_tools.py`) декорирован `@tool` и принимает `job_title: str`. Возвращает описание вакансии из словаря `_JD_TEMPLATES` по ключу (нормализованное название должности) или шаблонное описание по умолчанию `_DEFAULT_JD`.

3.6.15 Модуль `webhooks` (`routers/webhooks.py`) реализует in-memory реестр webhook-обратных вызовов.

Глобальный словарь `_registry: dict[str, str]` хранит соответствие `correlation_id => callback_url`.

Класс `WebhookRegisterRequest` содержит поля `correlation_id: str` и `callback_url: HttpUrl`. Класс `WebhookRegisterResponse` содержит поля `correlation_id`, `callback_url` и `registered: bool`. Класс `WebhookPayload` содержит поля `correlation_id`, `status` (строка "success" или "error"), `query_type`, `result` и `error`.

Маршрут `POST /register` сохраняет `callback_url` в `_registry` и возвращает `WebhookRegisterResponse`. Маршрут `DELETE /register/{correlation_id}` удаляет запись или возвращает 404. Маршрут

GET /register/{correlation_id} возвращает зарегистрированный URL или 404.

Функция `async _deliver(url, payload)` отправляет POST-запрос с телом `WebhookPayload` на `callback_url` с таймаутом 10 секунд, поглощает все исключения и логирует результат.

Функция `notify(correlation_id, *, job_status, query_type, result, error)` является публичным API для вызова из `orchestrator_service`. Проверяет наличие URL в `_registry`, формирует `WebhookPayload` и запускает `_deliver` как фоновую задачу через `asyncio.create_task (fire-and-forget)`.

3.6.16 Класс `AppConfig` (`core/config.py` сервиса `orchestrator-service`) наследуется от `BaseSettings`.

Поле `log_level` типа `str` определяет уровень логирования; по умолчанию – 'INFO'.

Поле `extraction_service_url` типа `str` задает базовый URL `extraction-service`.

Поле `storage_service_url` типа `str` задает базовый URL `storage-service`.

Поле `search_service_url` типа `str` задает базовый URL `search-service`.

Поле `auth_service_url` типа `str` задает базовый URL `auth-service`.

Поле `llm_api_url` и `llm_model` типа `str` задают URL и имя языковой модели для агентов `LangGraph`.

Поле `agent_timeout_sec` типа `float` определяет максимальное время выполнения одного графа; по умолчанию – 120.0.

Поле `max_retries` типа `int` определяет максимальное число повторных попыток `reflection`-узла; по умолчанию – 2.

Поле `checkpoint_backend` типа `str` задает бэкенд для `LangGraph Checkpointer`; поддерживаются значения 'memory' и 'redis'; по умолчанию – 'memory'.

Поле `redis_url` типа `str` задает URL Redis при использовании `redis`-бэкенда `checkpoint`; по умолчанию – 'redis://localhost:6379'.

3.6.17 Классы HTTP-клиентов (`services/clients/`) обеспечивают типизированное взаимодействие оркестратора с остальными сервисами. Вынесение клиентов в отдельный пакет, а не встраивание HTTP-логики непосредственно в инструменты `LangGraph`, позволяет тестировать инструменты независимо, подменяя клиенты заглушками, и переиспользовать один экземпляр клиента в нескольких инструментах без дублирования конфигурации.

Класс `ExtractionClient` (`services/clients/extraction_client.py`) является оберткой над `httpx.AsyncClient` с `base_url = settings.extraction_service_url` и таймаутом 60.0 секунд.

Метод `async process_resume(file_bytes: bytes, filename: str, correlation_id: str) => dict` формирует `multipart/form-data` запрос на

/extract/process, передает байты файла и его имя, добавляет заголовок X-Correlation-ID. Возвращает словарь с полем data из SuccessWrapper. При HTTP-ошибке выбрасывает ExternalServiceError с кодом ответа и телом.

Метод `async aclose()` => None закрывает внутренний HTTP-клиент; вызывается при завершении работы оркестратора. Все клиенты создаются единожды при инициализации оркестратора и используют пул соединений `httpx`, что исключает накладные расходы на установку TCP-соединения при каждом агентском вызове инструмента.

Класс `StorageClient` (`services/clients/storage_client.py`) является оберткой над `httpx.AsyncClient` с `base_url = settings.storage_service_url`.

Метод `async upsert(point: dict) => str` отправляет POST на `/storage/upsert` с JSON-телом точки, извлекает поле data из ответа и возвращает строковый идентификатор точки.

Метод `async batch_upsert(points: list[dict]) => list[str]` отправляет POST на `/storage/batch` и возвращает список идентификаторов сохраненных точек.

Метод `async get_by_id(point_id: str) => dict | None` выполняет GET на `/storage/{point_id}`. При статусе 404 возвращает None, при успехе – словарь точки.

Класс `SearchClient` (`services/clients/search_client.py`) является оберткой над `httpx.AsyncClient` с `base_url = settings.search_service_url`. Таймаут клиента устанавливается равным `settings.agent_timeout_sec + 10.0` секунд, чтобы HTTP-соединение не закрывалось раньше, чем завершится внутренний таймаут агента.

Метод `async hybrid_search(query: str, filters: dict, top_k: int, use_rerank: bool) => list[dict]` формирует объект `SearchRequest`, отправляет POST на `/search` и возвращает список словарей кандидатов из поля `data.candidates` ответа.

Класс `AuthClient` (`services/clients/auth_client.py`) является оберткой над `httpx.AsyncClient` с `base_url = settings.auth_service_url`.

Метод `async validate_token(token: str) => dict | None` отправляет GET на `/auth/validate` с заголовком `Authorization: Bearer <token>`. При статусе 200 возвращает JSON-payload токена, при статусе 401 или 403 – None.

3.6.18 Функция `make_checkpointer()` (`core/checkpointer.py`) создает экземпляр `LangGraph Checkpointer` в зависимости от значения `settings.checkpointer_backend`. При значении `'redis'` создает `AsyncRedisSaver` с подключением к `settings.redis_url`; при значении `'memory'` – `MemorySaver`. `Checkpointer` используется обоими графами (`build_search_graph` и `build_upload_graph`) для сохранения

промежуточных состояний между узлами и обеспечения идемпотентности при повторных вызовах.

Модуль `router` (`routers/orchestrator_router.py`) определяет HTTP-эндпоинты оркестратора. Маршрутизатор регистрируется с префиксом `/orchestrate` и подключается к приложению FastAPI в `lifespan`-обработчике после успешной инициализации `OrchestratorService`. Это гарантирует, что эндпоинты становятся доступными только после компиляции `LangGraph`-графов и проверки подключения ко всем зависимым сервисам.

Класс `OrchestratorRequest` (`models/request.py`) наследуется от `BaseModel` и содержит поля: `user_query: str` – запрос пользователя; `correlation_id: str` – идентификатор запроса, генерируется клиентом или BFF; `file_s3_key: str | None` – S3-ключ файла при загрузке; `filename: str | None` – оригинальное имя файла. Валидатор модели `validate_upload_fields` проверяет, что при `query_type == 'upload'` оба поля `file_s3_key` и `filename` не равны `None`, и выбрасывает `ValidationError` в противном случае.

Класс `OrchestratorResponse` (`models/response.py`) наследуется от `BaseModel` и содержит поля: `correlation_id: str`; `query_type: str`; `status: str` – значения 'success' или 'error'; `result: dict | None` – сводка результатов; `error: str | None` – сообщение об ошибке.

Маршрут `POST /orchestrate` принимает JSON-тело `OrchestratorRequest`. Формирует начальный `OrchestratorState`, запускает `orchestrator_service.run(state)` и возвращает `SuccessWrapper[OrchestratorResponse]`. При `AgentTimeoutError` возвращает `ErrorWrapper` со статусом 504.

Маршрут `GET /orchestrate/status/{correlation_id}` вызывает `orchestrator_service.get_status(correlation_id)` и возвращает `SuccessWrapper[dict]` с полями `correlation_id`, `values`, `next` и `done`. При отсутствии состояния возвращает `ErrorWrapper` со статусом 404. Маршрут используется BFF для поллинга статуса задачи, а также для отладки агентских сценариев: поле `next` позволяет определить, на каком узле граф ожидает продолжения выполнения.

3.7 Описание сервиса frontend

Сервис `frontend` реализует блок взаимодействия с пользователем и выступает в роли API Gateway системы. Он построен на фреймворке `NestJS` и использует архитектуру микрофронтон (Module Federation) для фронтенд-приложения. Это позволяет независимо разрабатывать и деплоить отдельные модули интерфейса (список кандидатов, чат, статистика, загрузка резюме), сохраняя при этом единый хост-приложение. Сервис обеспечивает приём запросов от пользователя, загрузку файлов, отображение ранжированных результатов с визуализацией степени соответствия, экспорт данных, а также потоковую передачу ответов через `WebSocket`.

3.7.1 Класс `OrchestratorProxy` (`backend-for-frontend/src/services/orchestrator.proxy.ts`) является `Injectable`-сервисом `NestJS`.

Поле `client` типа `AxiosInstance` является HTTP-клиентом, настроенным с `baseURL` из переменной окружения `ORCHESTRATOR_URL` (по умолчанию `http://localhost:8004`) и таймаутом 30000 мс.

Метод `async proxyUpload(fileBuffer, originalName, mimeType, jwt)` формирует объект `FormData` с файлом и его MIME-типом, отправляет `POST` на `/orchestrate/upload` с заголовком `Authorization: Bearer <jwt>` и возвращает объект `{ correlation_id: string }`.

Метод `async proxySearch(body, jwt)` отправляет `POST` на `/orchestrate/search` с телом запроса в `JSON` и заголовком авторизации. Возвращает ответ оркестратора без трансформации.

3.7.2 Класс `StorageProxy` (`backend-for-frontend/src/services/storage.proxy.ts`) является `Injectable`-сервисом `NestJS`.

Интерфейсы `CandidateVectorPayload` и `EmbeddingPoint` дублируют `Python`-модели хранилища для типизации `TypeScript`-кода.

Поле `client` типа `AxiosInstance` настроено с `baseURL` из переменной окружения `STORAGE_URL` (по умолчанию `http://localhost:8002`) и таймаутом 10000 мс.

Метод `async getEmbeddingPoint(pointId, jwt)` выполняет `GET`-запрос на `/storage/{pointId}`, извлекает поле `data` из обертки `SuccessWrapper` и возвращает объект `EmbeddingPoint`. При статусе 404 выбрасывает `NotFoundException`, при других ошибках – `InternalServerErrorException`.

Метод `async getCandidate(pointId, jwt)` является алиасом для `getEmbeddingPoint`.

3.7.3 Класс `UploadController` (`backend-for-frontend/src/controllers/upload.controller.ts`) обслуживает маршрут `POST /api/upload`. Использует декоратор `@UseInterceptors(FileInterceptor('file'))` для получения загружаемого файла. Извлекает `JWT` из куки `access_token` или `jwt` через приватный метод `extractJwt`. Передает буфер файла, оригинальное имя и MIME-тип в `proxy.proxyUpload`. При отсутствии файла выбрасывает `BadRequestException`, при отсутствии токена – `UnauthorizedException`.

3.7.4 Класс `CandidatesController` (`backend-for-frontend/src/controllers/candidates.controller.ts`) обслуживает маршрут `GET /api/candidates/:id`. Извлекает `JWT` из куки и передает идентификатор в `storage.getCandidate`. Контроллер является тонким слоем: он не преобразует данные и не реализует бизнес-логику, а только обеспечивает

аутентифицированный проксирующий доступ к storage-service. Ответ возвращается клиенту в том же формате, что и ответ storage-service, с единственным изменением – нижележащий HTTP-статус 404 транслируется в исключение `NotFoundException` NestJS.

3.7.5 Класс `WsGateway` (`backend-for-frontend/src/gateways/ws.gateway.ts`) реализует WebSocket-шлюз на базе Socket.io. Настроен на путь `/ws` с CORS `origin: '*'`.

Поле `server` типа `Server` является экземпляром Socket.io-сервера, внедряемым декоратором `@WebSocketServer()`.

Метод `handleConnection(client: Socket)` вызывается при подключении нового клиента; подписка выполняется отдельным сообщением.

Метод `handleDisconnect(client: Socket)` вызывается при отключении; очистка комнат выполняется Socket.io автоматически.

Метод `handleSubscribe(data: SubscribePayload, client: Socket)` обрабатывает сообщение `'subscribe'` с полем `correlation_id`, добавляет клиента в комнату с именем `correlation_id` через `client.join(correlation_id)` и возвращает `{ status: 'subscribed', correlation_id }`.

Метод `broadcast(payload: UpdatePayload)` принимает объект с полями `correlation_id`, `status` и `data`, и рассылает событие `'update'` всем клиентам в комнате с данным `correlation_id` через `server.to(correlation_id).emit('update', ...)`. Метод вызывается из webhook-эндпоинта оркестратора через NestJS EventEmitter или напрямую из HTTP-контроллера BFF, получающего callback от оркестратора. Комната Socket.io автоматически уничтожается, когда все клиенты покидают ее, поэтому явного управления жизненным циклом комнат не требуется.

3.7.6 Класс `SearchController` (`backend-for-frontend/src/controllers/search.controller.ts`) обслуживает маршруты поиска BFF.

Маршрут `POST /api/search` принимает JSON-тело и JWT из куки, проксирует запрос в `proxy.proxySearch(body, jwt)` и возвращает ответ оркестратора клиенту. При отсутствии токена выбрасывает `UnauthorizedException`. Использует валидационный pipe `ParseBodyPipe` для проверки обязательного поля `query`.

Маршрут `GET /api/status/:correlationId` выполняет GET-запрос на `ORCHESTRATOR_URL/orchestrate/status/{correlationId}` и возвращает JSON с полями `status` и `done` – предназначен для поллинга статуса задачи из SPA.

3.7.7 Класс `AppModule` (`backend-for-frontend/src/app.module.ts`) является корневым модулем NestJS BFF. Он не содержит бизнес-логики и служит исключительно точкой сборки всех провайдеров, контроллеров и

шлюза в единое приложение. Разделение на отдельные модули не применяется ввиду ограниченного числа компонентов BFF.

Импортирует `ConfigModule.forRoot({ isGlobal: true })` для загрузки переменных окружения. Импортирует `MulterModule.register({ limits: { fileSize: 20 * 1024 * 1024 } })` для ограничения размера загружаемых файлов. Регистрирует провайдеры `OrchestratorProxy` и `StorageProxy`. Регистрирует контроллеры `UploadController`, `SearchController` и `CandidatesController`. Регистрирует шлюз `WsGateway`. Среди провайдеров также зарегистрирован `HttpExceptionFilter` как глобальный фильтр исключений, форматирующий все NestJS-исключения в структуру `{ success: false, error: string, statusCode: number }`.

3.8 Описание сервиса `auth_service`

Сервис `auth_service` реализует блок обеспечения безопасности и контроля доступа. Он построен на базе Keycloak (OAuth2 + OpenID Connect) и обеспечивает единую точку аутентификации для всех пользователей и микросервисов системы. Сервис отвечает за аутентификацию кадровых специалистов, выдачу сервисных токенов, авторизацию межсервисных вызовов по ролям и детальный аудит всех операций.

3.8.1 Класс

`KeycloakStrategy` (`src/auth/strategies/keycloak.strategy.ts`) наследуется от `PassportStrategy(Strategy, 'jwt')` и реализует проверку JWT через JWKS-эндпоинт Keycloak.

Интерфейс `JwtPayload` описывает структуру `payload` токена: поле `sub` типа `string` – идентификатор пользователя; `preferred_username`: `string` – имя пользователя; `email`?: `string`; `realm_access`?: `{ roles: string[] }` – роли из Keycloak-реалма; `iat`: `number` и `exp`: `number` – временные метки.

Интерфейс `AuthUser` описывает нормализованный объект пользователя: поля `sub`, `username`, `email` и `roles`: `string[]`.

Конструктор вызывает родительский конструктор с конфигурацией: `jwtFromRequest` – извлечение токена из заголовка `Authorization: Bearer`; `secretOrKeyProvider` – провайдер JWKS через библиотеку `jwt-keycloak` с кешированием и ограничением 10 запросов в минуту, URI формируется из переменных окружения `KEYCLOAK_URL` и `KEYCLOAK_REALM`; алгоритм – RS256; `audience` – идентификатор клиента Keycloak.

Метод `validate(payload: JwtPayload): AuthUser` преобразует `payload` токена в объект `AuthUser`, извлекая роли из `realm_access?.roles` ?? `[]`. Если метод возвращает `null` или выбрасывает исключение, Passport автоматически отправляет ответ 401 `Unauthorized`. Кеширование JWKS снижает число запросов к Keycloak при высоком трафике: публичные ключи обновляются только при получении `kid`, отсутствующего в кеше.

3.8.2 Класс `AuditService` (`src/audit/audit.service.ts`) является Injectable-сервисом NestJS, реализующим сохранение событий аудита в базу данных через TypeORM.

Поле `logger` является экземпляром встроенного `Logger` NestJS, привязанным к классу `AuditService`.

Поле `repo` типа `Repository<AuditLog>` внедряется через декоратор `@InjectRepository(AuditLog)` и предоставляет TypeORM-репозиторий для работы с таблицей журнала аудита.

Метод `async logEvent(eventType, userId, correlationId, metadata)` создает запись `AuditLog` через `repo.create` и сохраняет ее через `repo.save`.

Параметр `eventType: string` задает тип события (например, `'login'`, `'upload'`); `userId: string | null` – идентификатор пользователя; `correlationId: string | null` – идентификатор запроса; `metadata?: Record<string, unknown>` – произвольные дополнительные данные. При ошибке сохранения логирует сообщение через `logger.error`, не пробрасывая исключение.

3.8.3 Класс `AuditLog` (`src/audit/audit-log.entity.ts`) является TypeORM-сущностью, отображаемой на таблицу `audit_log`. Хранение журнала аудита в реляционной базе данных (PostgreSQL) вместо файлов обусловлено необходимостью структурированного поиска по событиям: администратор может фильтровать записи по `user_id`, `event_type` и диапазону дат через стандартные SQL-запросы без парсинга лог-файлов.

Декоратор `@Entity('audit_log')` задает имя таблицы.

Поле `id` типа `number` декорировано `@PrimaryGeneratedColumn()` и является автоинкрементным первичным ключом.

Поле `event_type` типа `string` декорировано `@Column({ length: 64 })` и хранит тип события.

Поле `user_id` типа `string | null` декорировано `@Column({ nullable: true })` и хранит идентификатор пользователя.

Поле `correlation_id` типа `string | null` декорировано `@Column({ nullable: true })` и хранит идентификатор запроса.

Поле `metadata` типа `Record<string, unknown>` декорировано `@Column({ type: 'jsonb', nullable: true })` и хранит произвольные дополнительные данные в формате JSONB.

Поле `created_at` типа `Date` декорировано `@CreateDateColumn()` и автоматически устанавливается при создании записи.

3.8.4 Класс `AuthController` (`src/auth/auth.controller.ts`) обслуживает HTTP-эндпоинты аутентификации.

Маршрут `GET /auth/validate` декорирован `@UseGuards(AuthGuard('jwt'))`. Принимает запрос с заголовком

Authorization: Bearer <token>. Если стратегия KeycloakStrategy успешно валидировала токен – возвращает JSON { valid: true, user: AuthUser }. Если токен отсутствует или недействителен – Passport автоматически возвращает 401.

Маршрут GET /auth/me декорирован @UseGuards(AuthGuard('jwt')) и @Roles('recruiter', 'admin'). Извлекает объект AuthUser из request.user (установленного стратегией) и возвращает его в теле ответа.

Маршрут POST /auth/logout декорирован @UseGuards(AuthGuard('jwt')). Вызывает auditService.logEvent('logout', user.sub, null, {}) и возвращает { success: true }. Непосредственная инвалидация токена выполняется на стороне Keycloak – сервис лишь фиксирует событие.

3.8.5 Декоратор

@Roles(...roles) (src/auth/decorators/roles.decorator.ts) является пользовательским декоратором NestJS. Использует фабрику SetMetadata('roles', roles) для прикрепления списка допустимых ролей к обработчику маршрута. Используется совместно с классом RolesGuard.

Класс RolesGuard (src/auth/guards/roles.guard.ts) наследуется от интерфейса CanActivate. Конструктор внедряет Reflector для чтения метаданных маршрута. Метод canActivate(context: ExecutionContext): boolean извлекает список требуемых ролей через reflector.get('roles', ...). Если роли не заданы – возвращает true (маршрут открыт). Извлекает AuthUser из request.user и проверяет, содержит ли user.roles хотя бы одну из требуемых ролей через Array.prototype.some. При несоответствии выбрасывает ForbiddenException.

3.8.6 Класс AppModule (src/app.module.ts) является корневым модулем auth-service. Модуль намеренно сохраняется компактным: аутентификация является критической инфраструктурой, минимальное число зависимостей снижает время запуска и риск ошибок конфигурации.

Импортирует PassportModule.register({ defaultStrategy: 'jwt' }) для регистрации Passport. Импортирует TypeOrmModule.forRoot(...) с конфигурацией подключения к PostgreSQL: хост, порт и учетные данные берутся из переменных окружения через ConfigModule.forRoot. Импортирует TypeOrmModule.forFeature([AuditLog]) для регистрации сущности. Регистрирует провайдеры KeycloakStrategy, RolesGuard, AuditService. Регистрирует контроллер AuthController. Экспортирует PassportModule и RolesGuard для использования в других модулях при необходимости.

4 РАЗРАБОТКА ПРОГРАММНЫХ МОДУЛЕЙ

Настоящий раздел описывает программную реализацию ключевых модулей системы семантического поиска кандидатов. Система построена по микросервисной архитектуре и включает четыре Python-сервиса на базе фреймворка FastAPI, два TypeScript-сервиса на базе NestJS, а также набор разделяемых библиотек. Каждый сервис имеет собственную зону ответственности и взаимодействует с остальными компонентами посредством HTTP-вызовов, что позволяет масштабировать и обновлять их независимо. Схема программы представлена на чертеже ГУИР.400201.016 ПД.

4.1 Модули сервиса извлечения данных

4.1.1 Точкой входа сервиса является файл `main.py`, отвечающий за создание экземпляра FastAPI, настройку структурированного логирования через библиотеку `structlog` и регистрацию маршрутизатора с префиксом `/extraction`. Конфигурация выносится в отдельный модуль `core/config.py` на основе класса `AppConfig`, наследующего `BaseSettings` из Pydantic v2.

```
# apps/extraction-service/core/config.py
class AppConfig(BaseSettings):
    log_level: str = 'INFO'
    s3_endpoint: str
    s3_bucket: str = 'resumes'
    s3_secret_key: str
    s3_endpoint: str
    embedding_api_url: str
    embedding_model: str = "text-embedding-3-small"
    embedding_batch_size: int = 32
    chunk_size: int = 512
    chunk_overlap: int = 64
    llm_api_url: str
    llm_model: str
    ocr_engine: str = 'tesseract'
    max_file_size_mb: int = 20
    # ...

settings = AppConfig()
```

Наследование от `BaseSettings` позволяет Pydantic v2 автоматически считывать значения из переменных окружения и файла `.env`, выполняя валидацию типов при запуске. Значение `chunk_size = 512` является компромиссом между семантической полнотой фрагмента и ограничениями контекстного окна модели эмбедингов; `chunk_overlap = 64` обеспечивает перекрытие соседних фрагментов, исключая потерю информации на их границах. Если обязательное поле, например `s3_endpoint`, отсутствует в

окружении, приложение завершится с понятным сообщением об ошибке до обработки первого запроса.

4.1.2 Маршрутизатор сервиса реализует эндпоинт для загрузки и обработки файла резюме; принятый файл валидируется по расширению, сохраняется в объектном хранилище и передается в сервисный слой.

```
# apps/extraction-service/routers/extraction.py

ALLOWED_EXTENSIONS = {".pdf", ".docx", ".doc", ".png",
".jpg", ".jpeg"}
@router.post("/process-resume",
response_model=SuccessWrapper[ExtractionResponse])
async def process_resume_endpoint(
    file: UploadFile, x_correlation_id: str = Header(...),
) -> SuccessWrapper[ExtractionResponse]:
    ext = Path(file.filename).suffix.lower()
    if ext not in ALLOWED_EXTENSIONS:
        raise BadRequestError(f"Unsupported file format:
{ext}")
    file_bytes = await file.read()
    s3_key = await upload_to_s3(file_bytes, file.filename)
    document = await process_resume(file_bytes,
file.filename, s3_key)
    return
SuccessWrapper.of(ExtractionResponse(document=document))
```

Множество `ALLOWED_EXTENSIONS` выполняет роль первичной фильтрации до аллокации памяти под содержимое файла. Заголовок `x-correlation-id` объявлен обязательным для сквозного отслеживания запроса через цепочку сервисов. Исключение `BadRequestError` перехватывается глобальным обработчиком и преобразуется в HTTP 400 с телом `ErrorWrapper`, не требуя явных блоков `try/except` в коде маршрутизатора.

4.1.3 Функция `process_resume` реализует последовательный асинхронный конвейер из четырех этапов: разбор документа, извлечение профиля кандидата, нарезка текста и пакетная генерация эмбеддингов.

```
# apps/extraction-service/services/extraction_service.py
async def process_resume(file_bytes, filename, s3_key) ->
StructuredDocument:
    structured = await parse_document(file_bytes, filename)
    profile = await
extract_candidate_profile(structured.full_text)
    chunks = _chunk_text(structured.full_text,
settings.chunk_size, settings.chunk_overlap)
    embeddings = await generate_embeddings_batch(chunks)
```

```

        return StructuredDocument(source_s3_key=s3_key,
candidate_profile=profile, chunks=chunks, embeddings=embeddings,
...)

```

```

def _chunk_text(text: str, size: int, overlap: int) ->
list[str]:
    chunks, start = [], 0
    while start < len(text):
        chunks.append(text[start : start + size])
        start += size - overlap

    return chunks

```

Функция `parse_document` делегирует разбор в зависимости от формата: для PDF и DOCX используется библиотека `Docling`, для растровых изображений -- `pytesseract`. Извлечение именованных сущностей выполняется языковой моделью в режиме `Function Calling` с Pydantic-схемой `CandidateProfile` в качестве грамматики ответа, что исключает необходимость регулярных выражений. Алгоритм `_chunk_text` реализует скользящее окно с шагом `size - overlap`, а завершающий срез автоматически обрезается границей строки без генерации исключений.

4.1.4 Модели данных сервиса отражают иерархию страниц и блоков документа; корневым объектом результата обработки является `StructuredDocument`.

```

# apps/extraction-service/models/document.py
class DocumentBlock(BaseModel):
    block_type: str
    text: str
    page: int
    bbox: Optional[list[float]] = None

class StructuredDocument(BaseModel):
    source_s3_key: str
    full_text: str
    pages: list[DocumentPage] = []
    candidate_profile: Optional[CandidateProfile] = None
    chunks: list[str] = []
    embeddings: list[list[float]] = []
    metadata: dict = {}

```

Поля `chunks` и `embeddings` хранятся параллельными списками: `embeddings[i]` является векторным представлением фрагмента `chunks[i]`. Поле `candidate_profile` ссылается на модель из разделяемой библиотеки `libs/common`, исключая дублирование схем. Поле `bbox` в `DocumentBlock` заполняется только для PDF-документов с выраженной структурой -- при OCR и DOCX оно остается `None`.

4.2 Модули сервиса векторного хранилища и индексации

4.2.1 Сервис хранения обеспечивает сохранение и фильтрацию векторных представлений кандидатов в Qdrant. Qdrant выбран благодаря нативной поддержке полезной нагрузки рядом с вектором, гибким фильтрам по метаданным и открытой лицензии, позволяющей разворачивать хранилище как самостоятельный контейнер.

```
# apps/storage-service/models/vector.py
class CandidateVectorPayload(BaseModel):
    candidate_id: UUID
    skills: list[str] = []
    location: str | None = None
    years_of_experience: float | None = None
    chunk_index: int
    chunk_text: str
    indexed_at: datetime
    version: int = 1
    # ...

class EmbeddingPoint(BaseModel):
    id: UUID
    vector: list[float]
    payload: CandidateVectorPayload
```

Маршрутизатор `storage.py` экспортирует три эндпоинта: `POST /upsert` для одиночной записи, `POST /batch-upsert` для пакетного индексирования, `POST /filter` для фильтрации по метаданным без векторного поиска. Поле `candidate_id` связывает все фрагменты одного резюме в единый логический объект; поле `chunk_text` дублирует текст фрагмента в полезной нагрузке, позволяя кросс-энкодеру получать содержимое без запросов к объектному хранилищу. Семантика `upsert` гарантирует, что повторная загрузка резюме обновит существующие точки, не создавая дубликатов.

4.2.2 При запуске класс `CollectionService` проверяет наличие коллекции в Qdrant и создает ее с настроенными параметрами вектора и индексами полезной нагрузки при отсутствии.

```
# apps/storage-service/services/collection_service.py
async def ensure_collection(self) -> None:
    if await
self._client.collection_exists(self._collection):
    return
    await self._client.create_collection(
        collection_name=self._collection,
        vectors_config=VectorParams(size=1536,
distance=Distance.COSINE),
        hnsw_config=HnswConfigDiff(m=16, ef_construct=100),
```

```

    )
    for field in ["skills", "location", "languages"]:
        await self._client.create_payload_index(
            self._collection, field,
            PayloadSchemaType.KEYWORD
        )
    # числовые поля: years_of_experience (FLOAT),
    salary_expectation (INTEGER)

```

Косинусная метрика `Distance.COSINE` нечувствительна к длине вектора и оценивает угол между направлениями, что соответствует семантической близости текстовых эмбеддингов. Параметры `HNSW m=16` и `ef_construct=100` обеспечивают баланс между точностью поиска и объемом памяти. Без явных индексов `Qdrant` выполняет полный перебор полезной нагрузки, что неэффективно при больших коллекциях.

4.3 Модули сервиса семантического поиска

4.3.1 Сервис реализует гибридный конвейер, совмещающий векторный поиск в `Qdrant` с ре-ранжированием на основе кросс-энкодера. Весовые коэффициенты агрегации задаются в конфигурации сервиса.

```

# apps/search-service/core/models/request.py
class MetadataFilters(BaseModel):
    skills_required: list[str] = []
    skills_preferred: list[str] = []
    location: str | None = None
    min_experience_years: float | None = None
    max_salary: int | None = None
    languages: list[str] = []

class SearchRequest(BaseModel):
    query: str
    filters: MetadataFilters = MetadataFilters()
    top_k: int = 10
    use_rerank: bool = True
    correlation_id: str

class RerankScore(BaseModel):
    index: int
    score: float

```

Параметр `prefetch_multiplier = 3` задает коэффициент предварительной выборки: при `top_k = 10` из `Qdrant` извлекается 30 кандидатов для ре-ранжирования. Поля `skills` и `preferred_skills` разделены: обязательные формируют предусловие фильтрации в `Qdrant`, желаемые -- добавляют бонус к итоговой оценке. Весовой коэффициент `vector_weight = 0.6` отдает приоритет семантическому сходству над точным ранжированием кросс-энкодера.

4.3.2 Функция `run_search` реализует полный конвейер поиска; функция `_combine_scores` агрегирует векторные оценки и оценки кросс-энкодера по настраиваемой формуле.

```
# apps/search-service/services/search_service.py
async def run_search(req: SearchRequest) -> SearchResponse:
    query_vector = await embed_query(req.query)
    raw_hits = await vector_search(query_vector,
    req.filters, req.top_k * settings.prefetch_multiplier)
    if not raw_hits:
        return SearchResponse(total=0, candidates=[])
    rerank_scores = await rerank(req.query, raw_hits) if
req.use_rerank else None
    candidates = _combine_scores(raw_hits, rerank_scores,
    req.filters)
    candidates.sort(key=lambda c: c.final_score,
    reverse=True)
    return SearchResponse(total=len(candidates),
    candidates=candidates[: req.top_k])
    def _combine_scores(hits, rerank_scores, filters) ->
list[RankedCandidate]:
        for i, hit in enumerate(hits):
            v = _normalize(hit["score"], hits)
            r = rerank_scores[i] if rerank_scores else v
            final = min(settings.vector_weight * v +
settings.rerank_weight * r
                        + _business_bonus(hit["payload"],
filters), 1.0)
```

Запрос векторизуется той же моделью, что применялась при индексировании: использование разных моделей сделало бы косинусные расстояния бессмысленными. Функция `_combine_scores` реализует агрегацию оценок по формуле 4.1, которая задает вклад компонентов; бонус ограничен 0.10 и начисляется за совпадение предпочитаемых навыков и языков. Поле `explanation` в объекте кандидата передает рекрутеру текстовое разложение итоговой оценки.

$$final = 0.6 \cdot v_score + 0.4 \cdot r_score + bonus, \quad (4.1)$$

где *bonus* ограничен значением 0.10 и начисляется по правилам: начисляет по 0.02 балла за каждый совпавший предпочитаемый навык и по 0.03 при совпадении языка.

Нормализация векторных оценок выполняется методом `min-max` по набору результатов текущего запроса, что обеспечивает сопоставимость оценок независимо от плотности распределения в конкретной коллекции. Поле `explanation` формирует текстовое разложение итоговой оценки, которое передаётся на клиент для объяснимости результатов поиска.

4.3.3 Модуль ре-ранжирования инкапсулирует модель `CrossEncoder`; вызов делегируется в пул потоков через `run_in_executor`, предотвращая блокировку цикла событий `asyncio`.

```
# apps/search-service/services/reranker.py
@lru_cache(maxsize=1)
def _get_model() -> CrossEncoder:
    return CrossEncoder(settings.reranker_model)

    async def rerank(query: str, candidates: list[dict]) ->
list[float]:
    pairs = [[query, c["payload"].get("chunk_text", "")] for
c in candidates]
    raw_scores = await
asyncio.get_event_loop().run_in_executor(
        None, _get_model().predict, pairs
    )
    span = max(raw_scores) - min(raw_scores) or 1.0
    return [(s - min(raw_scores)) / span for s in
raw_scores]
```

Кросс-энкодер оценивает пары «запрос -- документ» совместно, что дает более высокую точность ранжирования по сравнению с косинусным сходством би-энкодера. Декоратор `lru_cache` гарантирует однократную загрузку весов модели. Защита `or 1.0` предотвращает деление на ноль при вырожденном множестве равных оценок.

4.4 Модули сервиса оркестрации

4.4.1 Сервис оркестрации координирует взаимодействие всех компонентов через агентные графы `LangGraph`. Для корректного управления ресурсами используется контекстный менеджер `lifespan` из `FastAPI`: метод `orchestrator_service.initialize()` создает чекпойнтер `LangGraph` на базе `PostgreSQL`, компилирует граф загрузки и граф поиска, устанавливает HTTP-клиенты для взаимодействия с другими сервисами. Метод `orchestrator_service.shutdown()` освобождает соединения при завершении работы контейнера. Компиляция графов выполняется один раз при старте, исключая накладные расходы на валидацию топологии в критическом пути обработки запросов.

4.4.2 Класс `OrchestratorState` является центральным объектом передачи данных внутри графов `LangGraph`; каждый узел получает полное состояние и возвращает новую версию с обновленными полями.

```
# apps/orchestrator-service/models/state.py
class OrchestratorState(BaseModel):
    user_query: str
```

```

        query_type: Literal["upload", "search", "batch",
"get_profile"] = "search"
        correlation_id: str
        auth_token: str
        extracted_document: Optional[dict] = None
        search_results: Optional[list[dict]] = None
        reflection: Optional[dict] = None
        retry_count: int = 0
        revised_query: Optional[str] = None
        max_retries: int = 2
        messages: list = Field(default_factory=list)
        error: Optional[str] = None

```

Поля `extracted_document` и `search_results` заполняются последовательно соответствующими узлами, реализуя паттерн неизменяемого состояния с явными переходами через `model_copy(update={...})`. Поля `retry_count` и `max_retries` ограничивают циклы повторного поиска; поле `messages` хранит историю ReAct-агентов и персистируется в PostgreSQL через чекпойнтер, позволяя возобновить сессию после перезапуска сервиса.

4.4.3 Рабочий процесс загрузки резюме реализован направленным ациклическим графом с линейным потоком `router -> extract -> store -> END`, где каждый узел обновляет определенные поля состояния.

```

# apps/orchestrator-service/graphs/resume_upload_graph.py
def build_upload_graph(checkpointer=None):
    builder = StateGraph(OrchestratorState)
    for name, fn in [("router", _router_node), ("extract",
extract_node), "store", store_node), ("error", error_handler)]:
        builder.add_node(name, fn)
    builder.set_entry_point("router")
    builder.add_edge("router", "extract")
    builder.add_edge("extract", "store")
    builder.add_edge("store", END)
    return builder.compile(checkpointer=checkpointer)

```

Узел `router` выполняет защитную проверку типа запроса; узел `extract` вызывает `extraction-service` и заполняет `extracted_document`; узел `store` выполняет пакетную запись в `storage-service`. Параметр `checkpointer` обеспечивает персистентность промежуточного состояния в PostgreSQL, позволяя возобновить обработку после сбоя с точки последнего завершенного шага.

4.4.4 Граф поиска реализует расширенный рабочий процесс с узлом рефлексии и условным ребром, позволяя агенту оценивать качество результатов и уточнять запрос при необходимости.

```

# apps/orchestrator-service/graphs/candidate_search_graph.py

```

```

def build_search_graph(checkpointer=None):
    builder = StateGraph(OrchestratorState)
    # ... добавление узлов: router, search, reflection,
format, error
    builder.add_conditional_edges("reflection",
should_retry, {"retry": "search", "done": "format"})
    return builder.compile(checkpointer=checkpointer)

async def reflection_node(state: OrchestratorState) ->
OrchestratorState:
    quality = "high" if len(state.search_results or []) >= 3
else "low" revised = await _llm_suggest_query(state.user_query,
state.search_results) \
    if quality == "low" else state.revised_query
    return state.model_copy(update={"reflection":
{"quality": quality}, "revised_query": revised})

def should_retry(state: OrchestratorState) ->
Literal["retry", "done"]:
    q = (state.reflection or {}).get("quality", "high")
    return "retry" if q == "low" and state.retry_count <
state.max_retries else "done"

```

Паттерн рефлексии позволяет агенту не только выполнять действие, но и оценивать его результат. При низком качестве `_llm_suggest_query` передает языковой модели исходный запрос и текущие результаты, получая альтернативную формулировку, охватывающую более широкий круг компетенций. Ограничитель `max_retries = 2` предотвращает бесконечные циклы при систематически низком качестве выдачи.

4.4.5 Языковая модель инициализируется через `lru_cache` и совместно используется всеми агентами; маршрутизирующий агент классифицирует тип запроса посредством структурированного вывода.

```

# apps/orchestrator-service/services/agents/
@lru_cache(maxsize=1)
def get_llm() -> ChatOpenAI:
    return ChatOpenAI(model=settings.llm_model,
openai_api_base=settings.llm_api_url, openai_api_key="not-used",
temperature=0)

class RouterOutput(BaseModel):
    query_type: Literal["upload", "search", "batch",
"get_profile"]

    async def route_query(user_query: str) -> str:
        result: RouterOutput = await
get_llm().with_structured_output(RouterOutput)\

        .ainvoke(ROUTER_PROMPT.format(user_query=user_query))
        return result.query_type

```

Параметр `temperature=0` устраняет случайность для задач классификации. Метод `with_structured_output` транслирует схему Pydantic в JSON Schema для Function Calling, гарантируя принадлежность результата одному из четырех допустимых типов. Метод `ainvoke` выполняет вызов асинхронно, не блокируя цикл событий.

4.4.6 Система вебхуков позволяет клиентам регистрировать URL обратного вызова и получать уведомления о завершении длительных операций в асинхронном режиме.

```
# apps/orchestrator-service/routers/webhooks.py
_registry: dict[str, str] = {} # correlation_id ->
callback_url

@router.post("/register")
async def register(correlation_id: str, callback_url: str) -
> dict:
    _registry[correlation_id] = callback_url
    return {"registered": True}
    async def notify(correlation_id: str, *, job_status: str,
query_type: str, result=None, error=None) -> None:
        url = _registry.pop(correlation_id, None)
        if url:
            asyncio.create_task(_deliver(url,
WebhookPayload(...)))
```

Метод `notify` извлекает запись с одновременным удалением (`dict.pop`), гарантируя однократную доставку. Доставка выполняется через `asyncio.create_task` без блокировки вызывающего кода; HTTP-клиент ограничен 10-секундным таймаутом, предотвращая накопление зависших соединений при недоступности клиентского URL.

4.5 Модули сервисов взаимодействия с пользователем и безопасности

4.5.1 Сервис аутентификации реализован на NestJS и делегирует проверку учетных данных серверу Keycloak по протоколу OIDC. Keycloak хранит учетные записи, определяет роли и выдает JWT-токены, снижая поверхность атаки -- `auth-service` не хранит паролей пользователей.

```
// apps/auth-service/src/auth/auth.controller.ts
@Controller('auth')
export class AuthController {
    @Post('login')
    @HttpCode(HttpStatus.OK)
    async login(@Body() dto: LoginDto) {
        return this.authService.login(dto.username,
dto.password);
    }
}
```

```

    }
    @Post('token/introspect')
    introspect(@Body() dto: IntrospectDto) {
        return this.authService.introspect(dto.token);
    }

    @Get('roles')
    @UseGuards(JwtAuthGuard, RolesGuard)
    @Roles('recruiter', 'admin', 'system')
    getRoles(@Request() req: { user: AuthUser }) {
        return { roles: req.user.roles };
    }
}

```

Метод `login` реализует поток `password grant` к Keycloak и возвращает пару токенов `access_token / refresh_token` без их хранения в базе данных. Метод `introspect` проверяет валидность токена через Introspection Endpoint, реализуя `stateless`-аутентификацию. Декоратор `@Roles` записывает допустимые роли в метаданные маршрута через `Reflector`, а `RolesGuard` сравнивает их с ролями из JWT при обработке запроса. Модуль `AppModule` конфигурирует `TypeORM` с параметром `synchronize`, отключенным в `production` для предотвращения автоматических миграций.

4.5.2 Сервис Backend-for-Frontend выступает прокси-слоем между браузерным клиентом и микросервисами, агрегируя данные и обрабатывая JWT без его хранения в JavaScript-контексте браузера.

```

// upload.controller.ts + orchestrator.proxy.ts
@Post('upload')
@UseInterceptors(FileInterceptor('file'))
async upload(@UploadedFile() file: Express.Multer.File,
@Req() req: Request) {
    return this.proxy.proxyUpload(file.buffer,
file.originalname, file.mimetype, extractJwt(req));}

    async proxyUpload(buffer: Buffer, name: string, mimetype:
string, jwt: string) {
        const form = new FormData();
        form.append('file', buffer, { filename: name, contentType:
mimetype });
        const { data } = await
axios.post(`${this.baseUrl}/orchestrate/upload`, form,
        { headers: { ...form.getHeaders(), Authorization:
`Bearer ${jwt}` } });
        return data;
    }
}

```

Утилита `extractJwt` читает токен из HTTP-only куки, исключая его хранение в JavaScript-контексте и снижая риск XSS-атак. Метод `proxyUpload`

упаковывает файл в `FormData` с корректным заголовком `Content-Type: multipart/form-data; boundary=...` и передает JWT в заголовке `Authorization`. Инкапсуляция логики проксирования в отдельном `injectable-сервисе` упрощает тестирование контроллера через мок-объекты.

4.6 Разделяемые библиотеки

4.6.1 Библиотека `libs/common` содержит канонические модели данных для всех Python-сервисов; централизация схем исключает «дрейф схем», при котором сервисы накапливают несовместимые определения одних и тех же сущностей.

```
# libs/common/models/candidate.py
class SkillEntry(BaseModel):
    name: str
    level: Optional[Literal["junior", "middle", "senior"]] =
None

class CandidateProfile(BaseModel):
    id: UUID = Field(default_factory=uuid4)
    full_name: Optional[str] = None
    email: Optional[str] = None
    experience: list[ExperienceEntry] = []
    skills: list[SkillEntry] = []
    raw_text_hash: str = ""

    @field_validator("email")
    @classmethod
    def validate_email(cls, v: str | None) -> str | None:
        if v and not re.match(r"^\[w.+-\]+@[w-]+\.[a-
z]{2,}$", v, re.I):
            raise ValueError("invalid email")
        return v
```

Поле `id` генерируется через `uuid4`, обеспечивая уникальность без внешней координации. Поле `raw_text_hash` содержит SHA-256 хэш текста резюме, позволяя избегать регенерации эмбедингов при повторной загрузке неизмененного документа. Декоратор `@classmethod` для валидатора обязателен в Pydantic v2 -- в отличие от v1, где методы могли быть обычными.

4.6.2 Для единообразного формата HTTP-ответов определены обертки `SuccessWrapper` и `ErrorWrapper` с дискриминирующим литералом `success`, а также иерархия исключений, отображающая HTTP-семантику.

```
# libs/common/models/wrappers.py + exceptions/base.py
class SuccessWrapper(BaseModel, Generic[T]):
    success: Literal[True] = True
    data: T
    @classmethod
```

```

def of(cls, data: T) -> "SuccessWrapper[T]":
    return cls(data=data)

class ErrorWrapper(BaseModel):
    success: Literal[False] = False
    error: str
    detail: Optional[str] = None

class ServiceException(Exception):
    status_code: int = 500

class BadRequestError(ServiceException): status_code = 400
class UnauthorizedError(ServiceException): status_code = 401
class NotFoundError(ServiceException): status_code = 404
class ExternalServiceError(ServiceException): status_code =
502

```

Дискриминирующий литерал `success` позволяет TypeScript-клиентам различать успешный и ошибочный ответ по единственному полю без анализа HTTP-статуса.

Глобальный обработчик `@app.exception_handler(ServiceException)` перехватывает любой подкласс и формирует ответ с кодом `exc.status_code` и телом `ErrorWrapper`. Фабричный метод `of` сокращает конструирование объекта до одной строки.

4.6.3 Библиотека `libs/langchain-utils` инкапсулирует фабрику чекпойнтера и шаблоны промптов для агентных графов.

```

# libs/langchain-utils/checkpointer.py + prompts/router.py
def make_checkpointer(dsn: str) -> AsyncPostgresSaver:
    return AsyncPostgresSaver.from_conn_string(dsn)

ROUTER_PROMPT = """\
Определи тип запроса и верни строго JSON без пояснений:
{"query_type": "<upload|search|batch|get_profile>"}
Правила: upload -- загрузка резюме; search -- поиск
кандидатов;
batch -- массовая загрузка; get_profile -- профиль
кандидата.
Запрос: {user_query}
"""

```

PostgreSQL выбран в качестве бэкенда чекпойнтера, поскольку уже присутствует в инфраструктуре и обеспечивает ACID-гарантии при записи промежуточных состояний. Функция возвращает неинициализированный объект: вызов `checkpointer.setup()` выполняется отдельно в `OrchestratorService.initialize`, явно разграничивая создание объекта и подготовку схемы таблиц. Инструкция «верни строго JSON без пояснений» совместно с `with_structured_output` образует двухуровневую защиту от неструктурированных ответов модели.

5 ПРОГРАММА И МЕТОДИКА ИСПЫТАНИЙ

Текст раздела

5.1 Подраздел

Текст раздела

6 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Текст раздела

6.1 Подраздел

Текст раздела

7 ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ СИСТЕМЫ ПОДБОРА ПЕРСОНАЛА С СЕМАНТИЧЕСКИМ АНАЛИЗОМ АГРЕГИРОВАННЫХ ДАННЫХ

7.1 Характеристика программного средства, разрабатываемого для реализации на рынке

Разрабатываемое программное средство представляет собой интеллектуальную систему подбора персонала с семантическим анализом агрегированных данных, предназначенную для автоматизации процессов поиска и оценки кандидатов в организации. Основная задача разработки – обеспечить точный и быстрый подбор релевантных соискателей на основе глубокого понимания компетенций, опыта и требований вакансий, сократить время скрининга резюме и снизить субъективные ошибки, характерные для традиционных методов подбора на основе ключевых слов. Потребность в таком решении обусловлена экспоненциальным ростом объёмов неструктурированных данных о кандидатах, высокой конкуренцией за квалифицированные кадры и необходимостью повысить эффективность HR-процессов в условиях цифровой трансформации рынка труда.

Разработка выполняется в рамках дипломного проекта и ориентирована на реализацию в виде самостоятельного продукта для открытого рынка. Система создаётся на базе открытых технологий и рассчитана на многократное использование разными компаниями и специалистами по подбору персонала без привязки к процессам одной конкретной организации.

Ожидаемым результатом внедрения для пользователей является значительное снижение трудозатрат рекрутеров на первичный скрининг, повышение релевантности рекомендаций кандидатов, удобная работа с данными на русском и других языках, получение прозрачного инструмента с объяснимыми результатами поиска по цене существенно ниже коммерческих аналогов. Для исполнителя результатом проекта выступает создание работоспособного прототипа рыночного продукта, демонстрация навыков работы с современными технологиями искусственного интеллекта и получение практического опыта разработки масштабируемых решений для сферы HR-технологий.

7.2 Расчет инвестиций в разработку программного средства для его реализации на рынке

При разработке программного продукта учитываются несколько основных направлений финансирования: оплата труда разработчиков, дополнительные выплаты, социальные обязательства, сопутствующие расходы, расходы на реализацию. Подробные расчёты по каждому пункту представлены далее.

7.2.1 Оплата труда разработчиков определяется исходя из численности специалистов, их должностных окладов и трудоёмкости выполняемых задач. Для расчёта применяется выражение, записанное в формуле 7.1.

Расчёт осуществляется по формуле

$$Z_o = K_{\text{пр}} \sum_{i=1}^n Z_{\text{чи}} \times t_i, \quad (7.1)$$

где $K_{\text{пр}}$ – коэффициент премий;

n – категория исполнителей, которые заняты разработкой;

$Z_{\text{чи}}$ – часовой оклад исполнителя i -ой категории, р.;

t_i – трудоёмкость работ исполнителя i -ой категории, ч.

Трудоёмкость выполнения работ определена с учётом масштаба и функциональной сложности интеллектуальной системы подбора персонала на основе многоагентной архитектуры с использованием LangGraph и LangChain. Для AI-инженера принято значение 320 часов, что отражает объём задач по проектированию и реализации оркестрации агентов, интеграции цепочек обработки данных, работе с семантическим поиском. Данный специалист отвечает за архитектуру, корректность бизнес-логики подбора кандидатов и интеграцию всех компонентов, что требует значительных временных.

Фронтенд-разработчику отведено 140 часов, поскольку реализация удобного пользовательского интерфейса для ввода запросов на естественном языке, визуализации ранжированных результатов с пояснениями релевантности и тепловыми картами совпадений является критически важным этапом для принятия системы пользователями. Кроме того, в его обязанности входит оптимизация UI/UX-элементов, адаптация под разные сценарии работы рекрутера и обеспечение интуитивной навигации, что также увеличивает трудоёмкость.

Тестировщику предусмотрено 100 часов. Этот объём времени необходим для подготовки тест-кейсов на разнообразные типы резюме и запросов, проведения функциональных, интеграционных и нагрузочных испытаний, включая проверку точности семантического поиска и стабильности агентов, а также документирования обнаруженных дефектов и контроля их исправления. Работа тестировщика обеспечивает надёжность системы, минимизируя риски некорректных рекомендаций кандидатов и сбоев при обработке реальных объёмов данных.

Дизайнеру выделено 80 часов, что связано с созданием интерактивных прототипов интерфейса, разработкой макетов экранов поиска, результатов и загрузки данных, а также проработкой визуальных решений для отображения степени соответствия кандидатов вакансии. Его задачи включают обеспечение удобства пользовательских сценариев, логичной навигации и визуальной целостности продукта, соответствующей современным требованиям к эргономике и эстетике HR-инструментов на базе искусственного интеллекта.

Затраты на основную заработную плату представлены в таблице 7.1.

Таблица 7.1 – Затраты на основную заработную плату

Категория исполнителя	Месячный оклад, р.	Часовой оклад, р.	Трудоемкость работ, ч	Итого, р.
AI-инженер	4 500	26,79	320	8 571,43
Фронтенд разработчик	2 400	14,29	140	2 000,00
Тестировщик	2 900	17,26	100	1 726,19
Дизайнер	2 200	13,09	80	1 047,62
Итого				13 345,24
Премия (55%) *				7 339,88
Всего затраты на основную заработную плату сотрудников				20 685,12

7.2.2 Формирование затрат на дополнительную заработную плату разработчиков. Дополнительная заработная плата представляет 10% от основной. В формуле 7.2 приведён расчет дополнительной заработной платы.

$$З_д = \frac{З_о \cdot Н_д}{100}, \quad (7.2)$$

где $З_о$ – основная заработная плата, р.;

$Н_д$ – норматив дополнительной заработной платы, %.

7.2.3 Формирование отчислений на социальные нужды. По состоянию на март 2026 года, норматив отчислений в ФСЗН и Белгосстрах представляет 34,6%, из которых 28% на пенсионное страхование, 6% на социальное страхование, 0,6% страховой тариф на обязательное страхование от несчастных случаев на производстве и профессиональных заболеваний. Учтём это значение и рассчитаем итоговые отчисления исходя из основной и дополнительной заработной платы. Вычисления представлены в формуле 7.3.

$$Р_{соц} = \frac{(З_о + З_д) \cdot Н_{соц}}{100}, \quad (7.3)$$

где $Н_{соц}$ – норматив социальных отчислений, %.

7.2.4 Формирование прочих расходов. Затраты на прочие расходы формируются из основной заработной платы и норматива 30% от заработной платы. Вычисления представлены в формуле 7.4.

$$Р_{пр} = \frac{З_о \cdot Н_{пр}}{100}, \quad (7.4)$$

где $Н_{пр}$ – норматив прочих расходов, %.

7.2.5 Формирование расходов на реализацию. Затраты на расходы на реализацию формируются из основной заработной платы и норматива 3% от заработной платы. Вычисления представлены в формуле 7.5.

$$P_p = \frac{Z_o \cdot H_p}{100}, \quad (7.5)$$

где H_p – норматив расходов на реализацию, %.

7.2.6 Формирование общей суммы затрат. В их состав входят расходы на оплату труда основной команды разработчиков, дополнительные выплаты, обязательные социальные отчисления, прочие расходы и расходы на реализацию. Общая сумма затрат определялась по формуле 7.6.

$$Z_p = Z_o + Z_d + P_{\text{соц}} + P_{\text{пр}} + P_p. \quad (7.6)$$

Таким образом, величина затрат на разработку программного средства указана в таблице 7.2.

Таблица 7.2 – Затраты на разработку программного средства

Название статьи затрат	Формула/таблица для расчета	Значение, р.
Основная заработная плата разработчиков	Таблица 1.1	20 685,12
Дополнительная заработная плата разработчиков	$Z_d = \frac{20\,685,12 \cdot 10}{100}$	2 068,51
Отчисление на социальные нужды	$P_{\text{соц}} = \frac{(20\,685,12 + 2\,068,51) \cdot 34,6}{100}$	7 872,76
Прочие расходы	$P_{\text{пр}} = \frac{20\,685,12 \cdot 30}{100}$	6 205,54
Расходы на реализацию	$P_p = \frac{20\,685,12 \cdot 3}{100}$	620,55
Общая сумма затрат на разработку	$Z_p = 20\,685,12 + 2\,068,51 + 7\,872,76 + 6\,205,54 + 620,55$	37 452,48

Общая сумма затрат составила 37 452,48 белорусских рублей.

7.3 Расчет экономического эффекта от реализации программного средства на рынке

Оценка экономического эффекта от реализации программного средства позволит определить его коммерческую привлекательность и потенциальную прибыльность. Эта величина зависит от прогнозируемого объема продаж, в

данном случае количества подписок, стоимости продукта и затрат на его разработку.

Для определения цены программного средства проведен анализ аналогичных решений на рынке в 2026 году. Большинство современных HR-платформ с такими элементами ИИ, как семантический поиск, анализ резюме, гибридный поиск, автоматизация скрининга, работают по модели SaaS-подписки (ежемесячная либо ежегодная оплата за пользователя или за компанию). В таблице 7.3 приведены примеры аналогичных продуктов и их стоимость за использование.

Таблица 7.3 – Стоимость использования продуктов-аналогов

Название продукта	Модель подписки	Цена, \$ за год
Talantix	Подписка за рекрутера на год	600
Huntflow	Подписка за рекрутера на год	950
Поток Рекрутмент	Подписка за рекрутера на год	815
Garmony.ai	Подписка за компанию на месяц	1 260
FriendWork	Подписка за компанию на месяц	650

Из представленной таблицы можно вычислить, что средняя стоимость подписки на год составляет 854,4 доллара США, что в пересчете на белорусские рубли составляет 2500 рублей.

Для расчета прироста чистой прибыли необходимо учесть налог на добавленную стоимость (НДС). Отпускная цена подписки на программное средство установлена на уровне 1900 рублей за одного рекрутера или малую компанию. Ожидается, что за год количество покупок подписки составит примерно 5. Ставка налога на добавленную стоимость, действующая на март 2026 года согласно законодательству Республики Беларусь, составляет 20%. Налог на добавленную стоимость рассчитывается по формуле:

$$\text{НДС} = \frac{C_{\text{отп}} \cdot N \cdot H_{\text{д.с}}}{100 + H_{\text{д.с}}}, \quad (7.7)$$

где N – количество оплат за подписку на программный продукт за год, шт.;

$C_{\text{отп}}$ – отпускная цена подписки программного средства, р.;

$H_{\text{д.с}}$ – ставка налога на добавленную стоимость, %.

Тогда налог на добавленную стоимость составляет:

$$\text{НДС} = \frac{1\,900 \cdot 5 \cdot 20}{100 + 20} = 1\,583,33 \text{ р.} \quad (7.8)$$

После вычисления налога на добавленную стоимость, можно рассчитать прирост чистой прибыли, который разработчик получит от

реализации программного продукта. Ставка налога на прибыль, в соответствии с действующим законодательством на март 2026 года, составляет 25%. Рентабельность продаж программного продукта установлена на уровне 20%. Формула для вычисления прироста чистой прибыли:

$$\Delta\Pi_{\text{ч}}^{\text{р}} = (\text{Ц}_{\text{отп}} \cdot N - \text{НДС}) \cdot \text{Р}_{\text{пр}} \cdot \left(1 - \frac{\text{Н}_{\text{п}}}{100}\right), \quad (7.9)$$

где $\text{Ц}_{\text{отп}}$ – отпускная цена подписки программного средства, р.;
 N – количество оплат за подписку на программный продукт за год, шт.;
 НДС – сумма налога на добавленную стоимость, р.;
 $\text{Р}_{\text{пр}}$ – рентабельность продаж;
 $\text{Н}_{\text{п}}$ – ставка налога на прибыль.
 Тогда прирост чистой прибыли составляет:

$$\Delta\Pi_{\text{ч}}^{\text{р}} = (1\,900 \cdot 5 - 1\,583,33) \cdot 20 \cdot \left(1 - \frac{25}{100}\right) = 118\,750,05 \text{ р.} \quad (7.10)$$

7.4 Расчет показателей экономической эффективности разработки и реализации программного средства на рынке

Для оценки экономической эффективности разработки и внедрения программного продукта на рынок, необходимо провести сравнительный анализ затрат на его разработку и ожидаемого прироста чистой прибыли за год. Если сумма затрат на разработку меньше годового экономического эффекта, то можно заключить, что инвестиции окупятся менее чем за один год.

Оценка эффективности инвестиций проводится с использованием расчета рентабельности инвестиций (англ. Return on Investment, ROI). Формула для расчета рентабельности инвестиций:

$$ROI = \frac{\Delta\Pi_{\text{ч}}^{\text{р}} - \text{З}_{\text{р}}}{\text{З}_{\text{р}}} \cdot 100\%, \quad (7.11)$$

где $\Delta\Pi_{\text{ч}}^{\text{р}}$ – прирост чистой прибыли, полученной от реализации программного средства на рынке, р;

$\text{З}_{\text{р}}$ – затраты на разработку и реализацию программного средства, р.

Тогда рентабельность инвестиций составляет:

$$ROI = \frac{118\,750,05 - 37\,452,48}{37\,452,48} \cdot 100\% = 217,07\%. \quad (7.12)$$

Данный уровень рентабельности подтверждает экономическую обоснованность реализации проекта и свидетельствует о его высоком потенциале для будущего роста и развития.

7.5 Вывод об экономической целесообразности реализации проектного решения

Проект по разработке интеллектуальную систему подбора персонала с семантическим анализом агрегированных данных продемонстрировал положительные результаты с точки зрения экономической целесообразности. Ожидаемый ROI составляет 217,07%, что указывает на высокую рентабельность и привлекательность инвестиций. Чистая прибыль, которую проект может генерировать ежегодно, составляет около 119 тыс. белорусских рублей. Эти показатели подтверждают, что проект обладает значительным потенциалом для получения прибыли в краткосрочной и среднесрочной перспективе.

Для привлечения большего числа пользователей, проект может предложить базовую бесплатную версию программного средства с ощутимым ограничением ресурсов (количество резюме и вакансий). Такой подход позволит привлечь малый и средний бизнес, дать возможность протестировать систему на реальных задачах и плавно перевести на платную подписку.

Чтобы выделиться и обеспечить конкурентоспособность, продукт должен сохранять и усиливать ключевые преимущества: существенно более низкую цену, полную прозрачность алгоритмов, встроенную мультиязычность для рынков СНГ и экспорта.

Кроме того, для долгосрочной конкурентоспособности необходим дополнительный функционал: интеграции с ведущими порталами (hh.ru, SuperJob), расширенная аналитика эффективности, инструменты снижения предвзятости, мобильное приложение для рекрутеров и кандидатов, а также готовые шаблоны для разных отраслей.

Основные риски связаны с необходимостью постоянного улучшения качества сопоставления, обеспечения скорости и надёжности при росте нагрузки, а также строгого соблюдения требований к защите персональных данных. При грамотном маркетинге и фокусе на уникальных сильных сторонах проект имеет все шансы выйти на самоокупаемость и занять устойчивую нишу в сегменте доступных ИИ-решений для подбора персонала.

ЗАКЛЮЧЕНИЕ

Должно быть минимум на 81 странице!!

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Каплан, А. Искусственный интеллект, цифровой мир и трансформация рынка труда / А. Каплан, М. Хенляйн // Цифровая экономика. – 2023. – № 4. – С. 12–25.
- [2] Мишра, С. Ограничения традиционных систем поиска персонала на основе ключевых слов / С. Мишра, Р. Гупта // Journal of HR Technology. – 2022. – Vol. 15. – P. 45–58.
- [3] Вульдрих, М. Многоагентные системы: введение / М. Вульдрих, Г. Вейсс // Многоагентные системы. – 2021. – С. 3–20.
- [4] Brown, T. Language Models are Few-Shot Learners [Электронный ресурс] / T. Brown [et al.] // arXiv preprint arXiv:2005.14165. – 2020. – Режим доступа: <https://arxiv.org/abs/2005.14165> – Дата доступа: 15.02.2026.
- [5] Vaswani, A. Attention Is All You Need [Электронный ресурс] / A. Vaswani [et al.] // Advances in Neural Information Processing Systems. – 2017. – P. 5998–6008. – Режим доступа: <https://arxiv.org/abs/1706.03762> – Дата доступа: 12.02.2026.
- [6] Shazeer, N. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer [Электронный ресурс] / N. Shazeer [et al.] // arXiv preprint arXiv:1701.06538. – 2017. – Режим доступа: <https://arxiv.org/abs/1701.06538> – Дата доступа: 18.02.2026.
- [7] Yang, A. Qwen Technical Report [Электронный ресурс] / A. Yang [et al.] // arXiv preprint arXiv:2309.16609. – 2023. – Режим доступа: <https://arxiv.org/abs/2309.16609> – Дата доступа: 20.02.2026.
- [8] Chase, H. LangChain [Электронный ресурс] / H. Chase. – Электронные данные. – Режим доступа: <https://github.com/langchain-ai/langchain> – Дата доступа: 22.02.2026.
- [9] LangChain AI. LangGraph Documentation [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://langchain-ai.github.io/langgraph/> – Дата доступа: 25.02.2026.
- [10] Lewis, P. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks [Электронный ресурс] / P. Lewis [et al.] // Advances in Neural Information Processing Systems. – 2020. – Vol. 33. – P. 9459–9474. – Режим доступа: <https://arxiv.org/abs/2005.11401> – Дата доступа: 27.02.2026.
- [11] Gao, Y. Retrieval-Augmented Generation for Large Language Models: A Survey [Электронный ресурс] / Y. Gao [et al.] // arXiv preprint arXiv:2312.10997. – 2023. – Режим доступа: <https://arxiv.org/abs/2312.10997> – Дата доступа: 01.03.2026.
- [12] Mikolov, T. Distributed Representations of Words and Phrases and their Compositionality [Электронный ресурс] / T. Mikolov [et al.] // Advances in Neural Information Processing Systems. – 2013. – P. 3111–3119. – Режим доступа: <https://arxiv.org/abs/1310.4546> – Дата доступа: 03.03.2026.
- [13] Devlin, J. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding [Электронный ресурс] / J. Devlin [et al.] // arXiv preprint

arXiv:1810.04805. – 2018. – Режим доступа: <https://arxiv.org/abs/1810.04805> – Дата доступа: 05.03.2026.

[14] Muennighoff, N. MTEB: Massive Text Embedding Benchmark [Электронный ресурс] / N. Muennighoff [et al.] // arXiv preprint arXiv:2210.07316. – 2022. – Режим доступа: <https://arxiv.org/abs/2210.07316> – Дата доступа: 06.03.2026.

[15] Malkov, Y. A. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs / Y. A. Malkov, D. A. Yashunin // IEEE Transactions on Pattern Analysis and Machine Intelligence. – 2018. – Vol. 42, № 4. – P. 824–836.

[16] Qdrant Team. Qdrant Vector Database Documentation [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://qdrant.tech/documentation/> – Дата доступа: 10.02.2026.

[17] Resume Matcher Project [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://github.com/srbhr/resume-matcher> – Дата доступа: 14.02.2026.

[18] Eightfold AI. Talent Intelligence Platform [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://www.eightfold.ai/> – Дата доступа: 17.02.2026.

[19] Phenom People. Talent Experience Management [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://www.phenompeople.com/> – Дата доступа: 21.02.2026.

[20] pdfplumber Documentation [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://github.com/jsvine/pdfplumber> – Дата доступа: 24.02.2026.

[21] Xu, Y. LayoutLM: Pre-training of Text and Layout for Document Image Understanding [Электронный ресурс] / Y. Xu [et al.] // Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. – 2020. – P. 1192–1201. – Режим доступа: <https://arxiv.org/abs/1912.13318> – Дата доступа: 28.02.2026.

[22] IBM Research. Docling: Document Understanding Toolkit [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://github.com/DS4SD/docling> – Дата доступа: 02.03.2026.

[23] Unstructured IO. Unstructured Open-Source Library [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://unstructured.io/> – Дата доступа: 04.03.2026.

Вычислительные машины, системы и сети: дипломное проектирование [Электронный ресурс]. – Электронные данные. – Режим доступа: https://www.bsuir.by/m/12_100229_1_136308.pdf – Дата доступа: 31.03.2026.

Экономика проектных решений: методические указания по экономическому обоснованию дипломных проектов [Электронный ресурс]. – Электронные данные. – Режим доступа: https://www.bsuir.by/m/12_100229_1_161144.pdf – Дата доступа: 31.03.2026.

ПРИЛОЖЕНИЕ А

(обязательное)

Исходный код программы

Код начинается с этого места;

001 Можно с нумерацией строк кода;

002 Шрифт может быть от 12 до 8 pt

ПРИЛОЖЕНИЕ Б
(обязательное)

Название приложения

ПРИЛОЖЕНИЕ В
(обязательное)

Отчет о проверке на заимствования

ПРИЛОЖЕНИЕ Г
(обязательное)

Спецификация

ПРИЛОЖЕНИЕ Д
(обязательное)

Ведомость документов